

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION V2 MASTER DOCUMENT

Campus Food Ordering • Scheduled Pickup • Vendor Operations • Super Admin Control

MASTER BUILD

Original V1 SRS: 25 pages preserved unchanged.

V2 Addendum: owner-confirmed decisions and additional V1 requirements.

Date: 25 August 2026

Non-removal guarantee: The original V1 document is included unchanged. The V2 addendum is additive and binding.

UNI8

GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

Campus Food Ordering • Scheduled Pickup • Vendor Operations • Super Admin Control

V1 MASTER DOCUMENT

This is the single global SRS for all UNI8 developers. It defines the product, roles, architecture, financial model, CRM, security requirements, development phases, handover requirements and final release standards.

LAUNCH COMMISSION RULE

At V1 launch, UNI8 commission is 8% of the applicable vendor sales amount. Vendor payable is therefore 92%. The 8% value is a Super Admin-controlled database setting and must never be hardcoded in application logic. Historical orders snapshot the rate applicable to them.

11 phases • 4 developers • 3-phase handovers • dedicated security phase • dedicated full-code debugging phase

Executive Roadmap

The system is divided into four development blocks. Developers 1–3 each own three phases. Developer 4 owns the final Security and Full Debugging phases. The handover package after Phases 3, 6 and 9 is mandatory and must contain everything required for the next developer to restore, understand, test and continue the project.

Developer 1	1–3	Foundation → Customer ordering/scheduling → Razorpay + Orders + QR
Developer 2	4–6	Vendor operations → Staff/restaurant operations → Payments/disbursements/vendor grievances
Developer 3	7–9	Super Admin → Customer 360 + central CRM → Analytics/settings/platform completion
Developer 4	10–11	Full security audit/remediation → Full code review, bug hunt and release certification

Critical product rule

Customer grievances are handled by UNI8 Super Admin/support only. They are never routed to a restaurant Vendor Admin. Vendor grievances are also handled by UNI8. This separation is a hard V1 requirement.

V1.0 • 25 August 2026

1. Product Definition

UNI8 is a university-campus food ordering platform designed around scheduled pickup. A student orders from one or multiple campus food joints, chooses pickup time(s), pays online through Razorpay, receives a unique QR pickup authorization for each restaurant order, and collects the food after staff verifies the QR.

1.1 Non-negotiable V1 rules

- Website only. No native app in V1.
- Customer must authenticate before ordering.
- Customer authentication uses phone number + OTP; first-time onboarding collects name, email and course. Do not collect block/hostel in V1.
- Razorpay is the customer payment method. No cash.
- UNI8 takes its configured commission from the order economics; the restaurant is owed the remainder according to the configured payable formula.
- Vendor payouts are manual in V1 and are executed by Super Admin. Vendor Admin only sees the payable/disbursement history and acknowledges receipt.
- Customer grievances ALWAYS go to UNI8 Super Admin/support. They never go to the restaurant/vendor admin.
- Vendor grievances ALWAYS go to UNI8 Super Admin/support. They never go to another vendor.
- Staff can only see their assigned restaurant's orders and scan pickup QRs.
- Vendor Admin can manage permitted product fields, stock and staff, but cannot manage platform-wide settings.
- Super Admin is the platform owner and has platform-wide control.
- Historical financial/order records must remain auditable even when restaurants/products/staff are archived.

1.2 UNI8 Commission & Vendor Payout Rule

At V1 launch, UNI8's platform commission is 8% of the vendor order sales amount. The vendor payout/payable is therefore 92% of the applicable vendor sales amount, subject to any separately approved adjustments/refunds recorded in the financial ledger. The 8% rate must NEVER be hardcoded in frontend or backend business logic. It must be stored as a Super Admin-controlled platform configuration value, with a default launch value of 8%.

EXAMPLE

Example: ■10,000 vendor sales → ■800 UNI8 commission (8%) → ■9,200 vendor payable. The website must visibly show the gross amount, commission rate, commission amount and resulting vendor payable wherever appropriate in Super Admin financial views and vendor payment views.

2. Product Enhancements Included in This SRS

Pickup-slot capacity	Prevent a restaurant from accepting more orders in a pickup window than it can handle.
Preparation cutoff	Prevent last-minute orders for slots that can no longer be prepared reliably.

Multi-restaurant pickup planner	One checkout can contain multiple restaurant orders with an explicit pickup sequence.
Walking-time matrix	Backend-configured walking time between restaurant pairs powers 'immediately after' scheduling.
Grace period	Defines operational handling after scheduled pickup time.
No-show state	Provides a clear auditable outcome for uncollected orders.
Order state machine	Prevents arbitrary/invalid status changes.
Immutable financial ledger	Makes customer payments, commission, vendor payable and manual disbursement traceable.
Admin audit log	Records privileged changes with actor, time, target and before/after context.
High-level Grievance CRM	Central support system for customers and vendors, owned by UNI8 Super Admins.
Customer 360 CRM	Gives Super Admin a complete operational view of each customer.
Restaurant workspace	Lets Super Admin inspect one restaurant at a time without cluttering the global navigation.
Access control centre	Lets Super Admin control Vendor Admin and Staff access, including reset/disable/force logout actions.
Vendor operational analytics	Shows only meaningful business and operational metrics.

2.1 Deliberately not in V1

- Delivery
- Timetable integration
- Native mobile apps
- Loyalty/rewards
- Referrals
- AI recommendations
- Public social feed
- Live map tracking
- Cash payments
- Automated vendor payouts
- Vendor-facing customer grievances
- Vendor self-service platform settings

3. Technology & Architecture

Framework	Next.js App Router + TypeScript.
Database	Supabase PostgreSQL as primary system of record.
Authentication	Supabase Auth. Customer phone OTP. Vendor/Admin/Staff role-controlled authentication.
Authorization	RBAC + restaurant-scoped access + PostgreSQL RLS. Server-side authorization checks are mandatory.
Storage	Supabase Storage for product images, grievance attachments and payout proof. Sensitive buckets private.
Payments	Razorpay for customer payment collection. Payment verification/webhooks are authoritative.
Backend	Next.js Server Actions/Route Handlers and server-side Supabase clients for privileged operations.
Analytics	Server-side aggregations/reporting queries; no full-table browser analytics.
Scalability	Target 20,000–25,000 accounts with headroom; designed around burst traffic during class breaks.
Observability	Application errors, payment events, audit events, grievance timeline and operational logs.

Current official Supabase guidance supports Next.js App Router with @supabase/ssr, recommends RLS for exposed application data, and documents Supavisor transaction pooling for serverless/short-lived application connections.

■cite■turn0search2■turn0search1■turn0search0■

4. Roles & Access Model

Customer	Own profile, orders, payments, QRs, ratings and grievances.
Vendor Admin	Assigned restaurant(s) only. Orders, analytics, permitted product management, payment visibility, scan, staff management and vendor-to-UNI8 grievances.
Staff	One assigned restaurant. Orders + Scan only. Maximum 5 active staff per restaurant.
Super Admin	Entire platform. Full customer, restaurant, vendor admin, staff, menu, product, order, payment, grievance, configuration and audit control.

Customer grievances are never routed to Vendor Admin. Vendor Admins can see only the grievance tickets they themselves raise to UNI8, not customer support tickets unless a Super Admin explicitly chooses to expose a specific operational message as part of an investigation.

5. Super Admin Information Architecture

The Super Admin interface must not present every possible restaurant control as one flat 12-item tab bar. Use a global workspace plus contextual entity workspaces.

5.1 Global Super Admin sidebar

COMMAND CENTER	Dashboard • Orders • Customers • Restaurants • Payments • Grievances • Analytics
PLATFORM	Staff & Access • Menu • Operations • Settings
CONTROL	Audit Log

5.2 Restaurant list

Restaurants is a global entity list. The Admin can search/filter/sort all restaurants and click one to enter that restaurant's dedicated workspace.

5.3 Restaurant workspace navigation

OVERVIEW	Dashboard
OPERATIONS	Orders • Products • Menu & Categories • Pickup & Capacity
PEOPLE & ACCESS	Vendor Admins • Staff
FINANCE	Payments • Disbursements
CUSTOMER EXPERIENCE	Grievances • Ratings
CONFIGURATION	Restaurant Settings • Walking Times
SYSTEM	Audit Log

6. Restaurant Workspace — Full Super Admin Control

Dashboard	Restaurant KPIs, current orders, upcoming pickup load, sales, AOV, collected/pending, outstanding vendor payable, staff count, vendor admins, open support issues and operational alerts.
Orders	All current/historical orders; search/filter; inspect customer, items, pickup, payment, QR, collection and grievance information; controlled operational corrections.
Products	View/edit/add/archive/restore products; price; image; description; cook time; category; order; stock status; availability.
Menu & Categories	Manage categories, product order, visibility and menu structure.
Pickup & Capacity	Restaurant hours, exceptions, slot capacity, preparation cutoff, preparation defaults and grace period.
Vendor Admins	Create/remove/disable Vendor Admins, assign restaurant scope, reset access, force logout, view access activity and manage credentials through the auth system.
Staff	Create/deactivate/reactivate up to 5 active staff, reset access, force logout, view activity and manage restaurant assignment.
Payments	Gross sales, UNI8 commission, vendor payable, amount already disbursed, outstanding amount, transaction ledger and payment exceptions.
Disbursements	Manual vendor payout records, amount, admin, timestamp, proof screenshot, covered payable, status and vendor acknowledgement.
Grievances	Customer and vendor tickets relating to the restaurant that are being handled by UNI8 Super Admins. Vendor Admin is not the ticket owner.
Ratings	Average rating, distribution, order-linked ratings, product-linked context and moderation controls.
Restaurant Settings	Name, branding, location, operating hours, status, pickup configuration, preparation defaults and operational notices.
Walking Times	View/edit walking-time relationship between this restaurant and every other configured pickup point.
Audit Log	All privileged actions associated with this restaurant.

7. Customer 360 CRM

Customers is a major global Super Admin area. It is not a simple user table. It is a high-level CRM where an Admin can understand the complete history and current state of an individual customer.

7.1 Customer list

Search	Name, phone, email, order ID, ticket ID.
Filters	Active/inactive, new, repeat, high-value, open grievance, cancellations, no-shows, joined date, last activity.
Columns	Customer, orders, lifetime spend, last order, average rating, open issues, status.
Quick view	Open customer snapshot without leaving the list.
Full view	Open Customer 360 workspace.

7.2 Customer 360 workspace

Overview	Customer identity, account status, lifetime orders, lifetime spend, AOV, completed, cancelled, no-show, average rating, open grievances and last activity.
Orders	Every order; full order details; restaurant, items, price snapshot, pickup, payment, QR, scan, staff, state history and linked grievance.
Activity Timeline	Chronological events: login/account events where appropriate, order placed, payment verified, QR issued, order prepared/ready, QR scanned, collected, rating, grievance, refund and other relevant platform events.
Payments	Razorpay payment references, amounts, statuses, timestamps, refunds and linked orders.
Grievances	Every customer ticket, current status, messages, attachments, assignment, resolution and reopen history.
Ratings	Every rating and comment, restaurant/product/order context.
Profile	Name, phone, email, course, account status, join date and permitted profile history.
Account & Security	Account status, last login, authentication events, active sessions where supported, force logout, disable/reactivate and access/security history.
Restaurant Affinity	Orders, spend and last order by restaurant.
Admin Notes	Internal-only customer support notes with author/timestamp and audit trail.

7.3 Customer operational flags

- High Value
- Frequent Customer
- Open Support Issue
- Payment Issue
- Repeated No-Shows
- Frequent Cancellations

Flags must be data-driven/operational, not arbitrary character judgments. Any admin-created flag must be auditable.

8. Super Admin Access & Credential Control

Super Admin has central control over Vendor Admin and Staff accounts. Passwords must never be stored or displayed in plaintext. The Admin can reset credentials through the authentication system rather than viewing existing passwords.

Create Vendor Admin	Yes
Assign Vendor Admin to restaurant	Yes
Disable/reactivate Vendor Admin	Yes
Reset Vendor Admin credential	Yes
Force logout Vendor Admin	Yes
View access/login activity	Yes, subject to provider capabilities
Create Staff	Yes
Assign Staff to restaurant	Yes
Disable/reactivate Staff	Yes
Reset Staff credential	Yes
Force logout Staff	Yes
Change restaurant assignment	Yes
Delete/erase authentication history	No — preserve audit/security history; use deactivation/archival.

Authentication should be managed through the auth provider; the platform must not invent its own plaintext-password storage. Supabase Auth supports phone authentication and server-side/session patterns for Next.js applications.

■cite■turn0search7■turn0search2■

9. Customer Website Requirements

Authentication

- Phone + OTP login before ordering.
- First login: name, email, course.
- Profile can later be edited within permitted fields.

Discovery

- Browse active restaurants.
- Search/filter restaurants/products.
- View restaurant status, hours, pickup availability.
- View product image, description, price and availability.

Cart & multi-restaurant

- Cart supports multiple restaurants.
- System partitions cart into restaurant orders.
- Customer chooses pickup sequence.
- First restaurant gets an explicit pickup time.
- Each subsequent restaurant gets a specific time OR 'immediately after previous pickup'.
- Immediate-after uses backend walking time.
- System validates restaurant hours, capacity, preparation constraints and feasibility.

Payment

- Razorpay only.
- Server-authoritative price/total.
- Payment verification + webhooks.
- No order confirmation based only on browser redirect.

Pickup

- Unique QR for each restaurant order.
- QR is single-use.
- QR validates restaurant, payment, status and collection state.
- Successful scan marks collected atomically.

Account

- Profile.
- Past/upcoming orders.
- Order details.
- Ratings 1–5 stars for eligible completed orders.
- Customer grievances go to UNI8 Super Admin/support only.

10. Vendor Admin Dashboard Requirements

Dashboard	GMV, orders, AOV, upcoming pickups, collected/pending, sales trend, pickup demand, top products and alerts.
Orders	All restaurant orders with search/filter/date/pickup/payment/collection status and full details.
Analytics	GMV trend; orders trend; AOV; orders by pickup period; top products; collected vs cancelled/no-show; repeat-customer share.
Products	Add/delete/edit product. Price required. Image/description/cook time optional. Out-of-stock persists until re-enabled.
Payments	Gross sales; UNI8 commission; applicable payment-processing cost; vendor payable; disbursed amount; outstanding payable; disbursement history; proof.
Scan Orders	Vendor Admin can scan QRs.
Manage Staff	Create/deactivate/reactivate up to 5 active staff; view credentials through secure reset flow; view activity.
Grievances	Vendor can raise grievances to UNI8 Super Admin/support, chat, attach evidence, see updates, status and resolution. Vendor cannot receive customer grievances.

11. Staff Portal

- Exactly two tabs: Orders and Scan.
- Orders limited to assigned restaurant.
- Shows upcoming orders, pickup times, items, payment/collection status.
- Scan verifies QR and marks eligible order collected.
- No product, price, payment, analytics, grievance or settings access.
- Maximum 5 active staff per restaurant.
- All scan and order actions are audit logged.

No frontend calculation may be trusted for financial truth. The backend/database financial calculation is authoritative.

All relevant financial screens must show: Gross Vendor Sales → UNI8 Commission Rate → UNI8 Commission Amount → Vendor Payable. Vendor payable at the launch rate is 92% of applicable sales.

The commission rate applicable to an order must be snapshotted onto the order/financial record at the time the financial obligation is created. Later changes to the global commission setting must NOT retroactively alter historical orders or already-created vendor payables.

Super Admin must be able to view the current commission rate, change it through an explicit controlled action, and see who changed it, when, the previous value, the new value and the reason in the audit log.

The Super Admin dashboard must contain a dedicated Commission Configuration control under Platform Settings. The launch/default value is 8%. This is a configurable database-backed setting, not a hardcoded constant.

11.5 Commission Configuration & Financial Display

12. Payments & Vendor Disbursement

V1 uses a two-sided financial flow: customer payment collection through Razorpay, followed by manual vendor payout by UNI8. The restaurant's due amount is the configured payable remainder after UNI8's commission and any explicitly configured deductions.

Customer payment	Razorpay payment for the trusted server-side checkout amount.
Verification	Server-side verification + webhook event handling.
Financial record	Create immutable order/payment/commission/vendor-payable records.
Vendor payable	Record the restaurant's due amount after configured UNI8 commission/deductions.

Outstanding	Aggregate unpaid vendor payable by restaurant.
Admin disbursement	Super Admin selects restaurant, sees payable amount, types amount to disburse, confirms, marks disbursed and uploads proof.
Partial payout	Allowed up to available payable; every payout records what it covers.
Vendor view	Disbursed amount, timestamp, proof, outstanding payable and history.
Vendor acknowledgement	Received / Not Received. Not Received creates/escalates a UNI8 support case.
Corrections	Financial corrections use reversal/adjustment events, not silent edits or deletion.

Razorpay's current documentation recommends webhooks for asynchronous payment events and server-side verification/API checks for critical confirmation. [cite](#) [turn0search0](#)

13. Central UNI8 Grievance CRM

All customer and vendor grievances are handled by UNI8 Super Admin/support. There is no customer-to-vendor grievance routing in V1.

Ticket	Unique ID; requester; customer/vendor role; restaurant if relevant; order/payment link; timestamps.
Categories	Payment, refund, wrong/missing item, pickup, QR, vendor issue, staff issue, product issue, account, technical, other.
Priority	Low, Normal, High, Urgent.
Status	Open, In Review, Waiting for Customer, Waiting for Vendor, Escalated, Resolved, Closed.
Assignment	Assign to Super Admin/support agent; reassignment history.
SLA	First-response and resolution timers; overdue highlighting.
Chat	Requester-visible conversation; admin-only internal notes.
Attachments	Private screenshots/images/documents.
Resolution	Closing requires resolution category + mandatory resolution note.
Reopen	Requester/admin can reopen with reason; timeline preserved.
Timeline	Immutable status/message/assignment/note/attachment/refund events.
Linked records	Order, payment, restaurant, product, staff, disbursement.
Refund workflow	Controlled refund request linked to original payment/order.
Search/filter	Ticket, person, restaurant, order, status, category, priority, assignee, date.
Templates	Approved response macros.
Escalation	Senior-admin/owner escalation with reason.
Customer satisfaction	Optional post-resolution CSAT.

14. Order & Pickup State Machine

cart	Temporary selection; no committed order.
payment_pending	Razorpay order exists; payment not verified.
paid	Payment verified/captured.
scheduled	Paid order has valid pickup time.
preparing	Vendor/staff has begun preparation.

ready_for_pickup	Order prepared.
collected	QR verified; food handed over.
cancelled	Cancelled under policy.
refund_pending	Refund initiated.
refunded	Refund confirmed.
no_show	Not collected within defined policy.

Only valid server-side transitions are permitted.

15. QR Pickup Security & Operations

- QR contains an opaque, non-guessable token.
- Server resolves token to order.
- Scanner verifies staff identity and restaurant scope.
- Payment must be verified.
- Order must belong to scanning restaurant.
- Order must not be cancelled/refunded/no-show/collected.
- Successful scan performs an atomic collection update.
- Second scan fails.
- Scan attempts are logged with staff, restaurant and timestamp.

16. Core Database Entities

profiles	Customer/staff/admin profile data.
roles / memberships	Role and restaurant-scoped access.
restaurants	Restaurant master/configuration.
restaurant_hours	Hours and exceptions.
restaurant_staff	Staff membership.
vendor_admin_memberships	Vendor Admin restaurant membership.
product_categories	Restaurant categories.
products	Product data and vendor-editable fields.
pickup_slots	Capacity and pickup-window data.
walking_times	Restaurant-to-restaurant walking-time matrix.
carts / cart_items	Temporary cart.
multi_order_groups	Multi-restaurant checkout grouping.
pickup_sequences	Selected restaurant order and calculated pickup times.
orders / order_items	Order header + immutable purchased-item snapshots.
payments / payment_events	Razorpay records and immutable event history.
vendor_payables	Order-level and aggregate vendor due amounts.
disbursements	Manual payout records + proof + acknowledgement.
ratings	Customer ratings.

grievance_tickets / messages / events	Central support CRM.
notifications	Notification records.
audit_logs	Privileged action history.
admin_settings	Platform configuration.

17. Security Baseline — Applies Throughout Development

- All exposed Supabase tables must use appropriate RLS policies.
- Customer, Vendor Admin and Staff data must be restaurant/user scoped.
- Super Admin permissions must be explicit and server-side.
- Never expose service-role/secret keys in browser code.
- Secrets must live in secure environment configuration.
- Never store plaintext passwords.
- Never trust client-submitted prices, totals, role IDs, restaurant IDs or payment status.
- Server must re-check authorization for every mutation.
- Private payout proofs/grievance attachments require controlled access.
- Rate-limit OTP, login, QR scan and sensitive support endpoints.
- Validate upload type/size.
- Use audit logging for privileged actions.
- Use idempotency for payment events, order creation and disbursements.
- Use soft deletion/archive where historical records exist.
- Security testing is a dedicated final phase and must also be considered during every earlier phase.

Supabase's current RLS guidance states that every table in an exposed schema should have RLS and that secret/service-role keys must never be exposed to browsers. [■cite■turn0search1■](#)

18. Scalability & Performance

- Support approximately 20,000–25,000 accounts with headroom.
- Design around peak concurrent order traffic during campus breaks.
- Index customer_id, restaurant_id, vendor_id, status, payment_status, pickup_time, created_at and grievance fields used in filters.
- Use composite indexes for operational queries such as restaurant + pickup time + status.
- Use pagination and server-side filtering for all large admin tables.
- Use server-side reporting/aggregation for analytics.
- Use connection pooling for transient/serverless traffic; Supabase documents Supavisor transaction pooling for this use case. [■cite■turn0search0■](#)
- Do not create a Realtime subscription for every order/customer by default.
- Load-test order bursts, payment webhooks and simultaneous QR scans.
- Monitor database connection usage and query performance.

19. Eleven-Phase Development Plan

The phases are intentionally divided into three 3-phase development blocks plus two final quality blocks. Each incoming developer receives the latest handover package and the same global SRS.

Developer 1 — Phases 1–3

PHASE 1 — Foundation, Architecture & Access

Objective: Create the production foundation of UNI8 and establish the platform's identity, database, authentication and role architecture.

Deliverables

- Next.js App Router + TypeScript application foundation.
- Supabase project integration.
- Core database schema and migrations for users, roles, restaurants, products and operational entities.
- Customer phone OTP authentication and onboarding.
- Vendor Admin, Staff and Super Admin authentication foundations.
- RBAC and restaurant-scoped authorization model.
- RLS foundations for all relevant tables.
- Shared UI/design system and route structure for all four experiences.
- Storage structure for product images, grievance attachments and payout proofs.
- Environment/secrets structure and development/staging conventions.
- Base audit-log framework.

Phase completion standard

- All four role experiences have authenticated route foundations.
- Roles cannot access unauthorized areas.
- Database migrations are reproducible.
- RLS/authorization foundation exists before feature expansion.
- Project can be run by a new developer using only the documented setup.

PHASE 2 — Customer Discovery, Cart & Scheduling

Objective: Build the complete pre-payment customer ordering experience, including multi-restaurant scheduling.

Deliverables

- Customer home/explore experience.
- Restaurant listing and search.
- Restaurant menu/product pages.
- Product availability and out-of-stock handling.
- Cart with multiple restaurants.
- Restaurant-specific cart grouping.
- Pickup sequence selection.
- First pickup time selection.
- Immediate-after scheduling.
- Walking-time matrix integration.
- Restaurant hours, preparation time, pickup capacity and preparation cutoff validation.
- Grace-period data model.
- Checkout summary and order preview.

Phase completion standard

- Customer can discover restaurants and products.
- Customer can create a valid single- or multi-restaurant cart.
- Pickup times are calculated correctly.
- Invalid/over-capacity/closed slots are prevented.
- Multi-restaurant sequence is persisted correctly.

PHASE 3 — Razorpay, Orders & QR Pickup

Objective: Complete the customer transaction lifecycle and restaurant pickup verification.

Deliverables

- Razorpay customer checkout integration.
- Server-side trusted order total calculation.
- Payment verification and webhook handling.
- Idempotent payment event processing.
- Order creation and order state machine.
- Customer order history and order detail.
- Unique QR/pickup token generation.
- Vendor/Staff scanning and atomic collection.
- QR invalidation after collection.
- Payment/order exception handling.
- Customer rating flow for eligible completed orders.

Phase completion standard

- Successful payment creates correct paid order(s).
- Failed/duplicate payment events do not create duplicate orders.
- Each restaurant order receives its own valid QR.
- Only authorized restaurant staff/vendor can collect.
- Double scanning cannot double-collect.
- Customer can see completed/upcoming orders and rate eligible orders.

MANDATORY HANDOVER 1 — After Phase 3

Developer 1 must provide the complete handover package before leaving the project.

Source ZIP	Complete project source, configuration templates, migrations, assets and all files required to run/build the current system.
Environment template	Complete list of required environment variables with descriptions and safe placeholder values; NEVER include real secrets.
Database package	All SQL migrations, schema definitions, indexes, RLS policies, functions/triggers and seed data required to reproduce the current database.
README	Exact setup, install, environment setup, local run, build and deployment instructions.
Architecture document	Current architecture, route structure, major modules, data flow and external integrations.
Auth/RBAC document	Roles, permissions, restaurant scoping and authentication behaviour.
API/integration document	Endpoints/server actions, request/response expectations, webhook behaviour and third-party integration notes.

Payment document	Razorpay flow, event handling, verification logic and test-mode setup.
Known issues	Every known bug, limitation, unfinished item and workaround.
Test report	What was tested, test accounts, test scenarios, pass/fail status and unresolved failures.
Database backup/export	Current non-secret development/staging database schema/data export sufficient to restore the environment, subject to privacy policy.
Deployment notes	Current deployment configuration and exact steps for redeployment.
Third-party inventory	All packages, services, SDKs, fonts/assets and licenses used.
Handover notes	What changed in Phases 1–3, current state, next tasks and anything a new developer must know.

Developer 2 — Phases 4–6

PHASE 4 — Vendor Admin Operations

Objective: Build the complete Vendor Admin operational dashboard.

Deliverables

- Vendor Dashboard.
- Orders page with filters/search/statuses.
- Vendor analytics and required business graphs.
- Products page with price, image, description, cook time and out-of-stock controls.
- Vendor Scan Orders.
- Manage Staff.
- Maximum 5 active staff per restaurant.
- Vendor Admin access/profile controls.
- Restaurant-scoped data isolation.

Phase completion standard

- Vendor can see only assigned restaurant data.
- Vendor can manage permitted product fields.
- Out-of-stock persists until re-enabled.
- Vendor analytics reconcile with orders.
- Staff creation/deactivation works within limits.

PHASE 5 — Staff Portal & Restaurant Operations

Objective: Build the staff-facing operational system and strengthen restaurant-level pickup operations.

Deliverables

- Staff Orders page.
- Staff Scan page.
- Upcoming order visibility.
- Pickup time visibility.
- Operational order statuses.
- Restaurant pickup-capacity controls.

- Preparation cutoff controls.
- Grace period/no-show handling.
- Restaurant operating hours/exceptions.
- Restaurant-specific operational alerts.

Phase completion standard

- Staff has only Orders + Scan permissions.
- Staff cannot access vendor finances or customer information beyond what is operationally required.
- Capacity/cutoff rules prevent invalid customer promises.
- No-show and late pickup states are recorded correctly.

PHASE 6 — Payments, Manual Disbursement & Vendor Grievances

Objective: Build the full V1 vendor financial visibility and vendor-to-UNI8 support workflow.

Deliverables

- Vendor Payments page.
- Vendor payable calculations.
- Outstanding payable view.
- Per-order financial breakdown.
- Manual disbursement queue in Super Admin.
- Admin selects vendor, sees amount available, enters amount, marks disbursed and uploads proof.
- Partial disbursement support.
- Vendor disbursement history.
- Vendor proof viewing.
- Vendor Received / Not Received acknowledgement.
- Not Received escalation into UNI8 grievance CRM.
- Financial audit events.
- Vendor grievance creation and messaging to UNI8 Super Admin.

Phase completion standard

- Vendor payable reconciles with paid orders and commission logic.
- Admin cannot accidentally disburse more than valid payable without an audited override.
- Vendor sees exact disbursement information.
- Proof upload/access is secure.
- Vendor grievances reach UNI8 only.

MANDATORY HANDOVER 2 — After Phase 6

Developer 2 must deliver a complete updated handover package, not merely a list of changed files.

Complete ZIP	Entire current project including all prior and current phases.
Change report	Detailed summary of Phases 4–6 and important changes to earlier code.
Updated environment template	All new variables and third-party configuration requirements.

Updated database package	All migrations/policies/functions and current schema state.
Updated architecture	Current route/module/data-flow architecture.
Payment/disbursement documentation	Exact vendor payable calculation, manual payout flow, proof handling and acknowledgement logic.
RBAC matrix	Current permissions for Customer, Vendor Admin, Staff and Super Admin.
Test evidence	Vendor, Staff, payment, disbursement and grievance scenarios tested.
Known issues	Complete unresolved bug/limitation list.
Restore/setup guide	A new developer must be able to restore and run the exact handover state.
Dependency/license inventory	Updated packages/services/assets and versions.
Deployment state	Current deployed environment and redeployment instructions.
Next-phase brief	What Phases 7–9 must accomplish and known risks.

Developer 3 — Phases 7–9

- Vendor/Admin financial views clearly expose gross sales, commission rate, commission amount and vendor payable.
- Historical orders snapshot the commission rate used for that order so future configuration changes do not rewrite historical financials.
- Vendor payable is calculated and displayed as gross vendor sales minus the configured UNI8 commission; at launch this means 92% payable.
- Super Admin financial configuration includes the 8% launch commission as a configurable database-backed setting, not a hardcoded value.

PHASE 7 — Super Admin Command Center & Restaurant Workspaces

Objective: Build the platform owner's complete global control system and contextual restaurant workspace.

Deliverables

- Global Super Admin Dashboard.
- Global Orders.
- Global Restaurants list.
- Restaurant creation, archive/deactivate/reactivate.
- Full restaurant workspace with grouped contextual navigation.
- Restaurant Dashboard.
- Restaurant Orders.
- Restaurant Products.
- Menu & Categories.
- Pickup & Capacity.
- Vendor Admins.
- Staff.
- Restaurant Payments.
- Disbursements.
- Restaurant Grievances.
- Ratings.

- Restaurant Settings.
- Walking Times.
- Restaurant Audit.
- Super Admin control over all supported restaurant/vendor/staff data.
- Super Admin access/credential control for Vendor Admin and Staff.

Phase completion standard

- Super Admin can enter any restaurant and see its complete operational picture.
- Restaurant workspace is contextual and not a flat cluttered tab list.
- Admin can add/edit/archive restaurants.
- Admin can manage vendor admins and staff access.
- Admin can edit products/prices/images/availability.
- All privileged actions are audited.

PHASE 8 — Customer 360 CRM & Central Grievance CRM

Objective: Build the high-level CRM and central support operation for UNI8.

Deliverables

- Global Customer directory.
- Customer search/filter.
- Customer 360 workspace.
- Customer overview snapshot.
- Customer order history.
- Customer activity timeline.
- Customer payments.
- Customer grievances.
- Customer ratings.
- Customer profile.
- Customer account/security.
- Restaurant affinity.
- Admin-only customer notes.
- Central customer grievance queue.
- Central vendor grievance queue.
- Ticket assignment.
- Priority.
- SLA.
- Chat.
- Internal notes.
- Attachments.
- Resolution notes.
- Reopen.
- Escalation.
- Linked orders/payments/disbursements.

- Refund workflow records.
- Support templates.
- Customer satisfaction.

Phase completion standard

- Admin can understand a customer's complete operational history from one workspace.
- Customer grievances are visible and actionable only to UNI8 support/Super Admins.
- Vendor Admins cannot receive customer grievances.
- Every ticket has a complete auditable timeline.
- Closing requires a resolution note.
- Reopening preserves history.

PHASE 9 — Global Analytics, Settings, Audit & Platform Completion

Objective: Complete the platform-wide management layer and bring all features into a coherent production-ready V1.

Deliverables

- Global Analytics.
- Restaurant comparison analytics.
- Platform GMV/order/AOV analytics.
- Customer retention/repeat-order metrics.
- Pickup demand analytics.
- Product performance analytics.
- Grievance performance analytics.
- Global Staff & Access centre.
- Global Menus.
- Global Operations settings.
- Platform Settings.
- Walking-time matrix management.
- Commission configuration.
- Pickup capacity configuration.
- Preparation/grace policies.
- Notification templates where applicable.
- Global Audit Log.
- Search/filter/export capabilities where appropriate.
- Cross-module reconciliation.
- Admin dashboards and operational alerts.
- End-to-end role workflows.

Phase completion standard

- All major platform functions are reachable through the intended global/contextual IA.
- Analytics reconcile with source data.
- Settings affect the correct modules.

- Audit log captures privileged actions.
- Customer, Vendor Admin, Staff and Super Admin end-to-end journeys work together.

MANDATORY HANDOVER 3 — After Phase 9

Developer 3 must deliver the final development handover. Developer 4 will use this exact package for Security and Full Debugging.

Complete source ZIP	Full project, migrations, configuration templates, assets and deployment files.
Full database package	All migrations, RLS, functions/triggers, indexes, seed data and restore instructions.
Complete architecture document	Final route/module/data-flow architecture and system boundaries.
Complete permissions matrix	Every role × every route/action/data domain.
Payment documentation	Customer payment, webhook, commission, payable, disbursement and acknowledgement flows.
CRM documentation	Customer 360 structure and grievance lifecycle.
Operational documentation	Restaurant capacity, pickup scheduling, walking times, no-show/grace rules.
Audit documentation	What is logged, where, how to query and retention expectations.
Environment/secrets template	No real secrets; all variables documented.
Deployment runbook	Exact build/deploy/rollback instructions.
Test suite/report	All automated/manual tests, coverage where available, test accounts and known failures.
Known issues register	Every known bug, technical debt item, warning and limitation.
Dependency/SBOM-style inventory	Packages, versions, services, licenses and external assets.
Database backup/export	Current restorable non-secret environment backup/export.
Performance report	Observed performance, slow queries, load-test results and bottlenecks.
Security status	Known security concerns already identified and their status.
Final developer handover README	How Developer 4 should start, what to inspect first and what must not be changed without understanding dependencies.

PHASE 10 — Full-Website Security Audit & Remediation

Objective: Perform a dedicated security phase across the ENTIRE website and fix every security issue found as a release blocker. This is not a documentation-only audit.

Deliverables

- Audit Customer dashboard and every customer endpoint/action.
- Audit Vendor Admin dashboard and every vendor endpoint/action.
- Audit Staff dashboard and scanner.
- Audit Super Admin dashboard and every privileged action.
- Audit Supabase RLS policies table-by-table.
- Audit authentication and session handling.
- Audit OTP abuse/rate limits.
- Audit password/reset/credential flows for staff/vendor/admin.
- Audit role escalation and privilege boundaries.
- Audit restaurant-scoping isolation.
- Audit Customer 360 privacy boundaries.
- Audit grievance privacy and attachment access.

- Audit payout proof storage/access.
- Audit Razorpay integration, webhook authenticity, idempotency and payment-state trust.
- Audit QR generation/verification/replay/double-scan protection.
- Audit input validation and output encoding.
- Audit file uploads.
- Audit XSS/CSRF/SQL injection/IDOR/BOLA-style access control risks.
- Audit server actions and API routes.
- Audit environment variables and secrets.
- Audit dependency vulnerabilities.
- Audit logging for sensitive actions.
- Audit rate limiting and abuse controls.
- Audit error messages for information leakage.
- Audit security headers/cookies/session configuration.
- Perform penetration-style manual testing across roles.
- Fix every discovered security issue appropriate to the release.
- Re-test all fixes and confirm no regression.

Phase completion standard

- Security report exists with findings, severity, evidence, remediation and retest result.
- Critical/high security issues are fixed before release.
- No known critical authentication, authorization, payment, QR, RLS or data-isolation vulnerability remains.
- Cross-restaurant/customer data access tests pass.
- Secrets are not exposed in source/build/browser.
- Payment and payout controls pass security testing.
- Security regression tests exist for fixed issues.
- Release is blocked if unresolved critical/high issues remain.

PHASE 11 — Full Code Review, Bug Hunt & Release Certification

Objective: Inspect the complete codebase line-by-line/module-by-module for possible bugs, edge cases, race conditions, broken states and production failures. Fix issues rather than merely reporting them.

Deliverables

- Review every application module.
- Review every server action and API route.
- Review every database query and mutation.
- Review every RLS policy.
- Review every state transition.
- Review payment/webhook edge cases.
- Review duplicate-click/retry/refresh behaviour.
- Review simultaneous QR scan behaviour.
- Review multi-restaurant scheduling edge cases.
- Review pickup capacity race conditions.

- Review stock/out-of-stock race conditions.
- Review manual disbursement race conditions and duplicate payouts.
- Review grievance state transitions.
- Review admin role changes and access revocation.
- Review file upload failures.
- Review network failure/retry states.
- Review empty/loading/error states.
- Review responsive layouts and browser compatibility.
- Review analytics calculations.
- Review financial reconciliation.
- Review timezone/date/time-slot logic.
- Review archived/deactivated entity behaviour.
- Review database constraints and orphan-record risks.
- Run full regression suite.
- Run production build.
- Run end-to-end acceptance tests.
- Fix all discovered bugs.
- Repeat regression after fixes.
- Produce final release certification.

Phase completion standard

- Every known bug found in the phase is either fixed or formally accepted as a non-blocking issue with owner and reason.
- No critical/high production bugs remain.
- Payment, order, QR, access, grievance and payout flows pass end-to-end regression.
- Production build succeeds cleanly.
- Final handover package is complete.
- Final release recommendation is explicitly documented as GO / NO-GO.

22. Final Release Package After Phase 11

Production source ZIP	Complete final source code and all required project files.
Database migrations	Complete reproducible migration history.
Database restore/export	Approved backup/export and restore procedure.
Environment variable template	All required variables without real secrets.
Deployment runbook	Build, deploy, rollback, environment setup and emergency procedures.
Architecture documentation	Final system architecture and module map.
Permissions/RBAC matrix	Final role/action/data-access matrix.
API/integration documentation	Razorpay, Supabase and any other integrations.
Payment/reconciliation guide	How to verify customer payments, vendor payables and disbursements.
CRM operations guide	Customer 360 and grievance workflows.
Admin operations guide	Restaurant, staff, vendor admin, payments, audit and support workflows.
Security report	Findings, fixes, retests and residual risk.

Bug/debug report	Bugs found, fixes, regression results and accepted non-blockers.
Test suite/report	Automated/manual tests and final results.
Dependency/license inventory	Versions and licenses.
Monitoring/observability guide	Logs, alerts, health checks and operational dashboards.
Known limitations	Explicit list of anything intentionally deferred to V2.
Emergency recovery guide	What to do for payment mismatch, database incident, credential compromise, QR abuse or major outage.

23. Global Acceptance Criteria

- At launch, the configured UNI8 commission rate is 8%.
- A ₹10,000 applicable vendor sales amount produces ₹800 UNI8 commission and ₹9,200 vendor payable at the 8% rate.
- The 8% rate is not hardcoded anywhere in frontend or backend business logic; it is read from Super Admin-controlled configuration.
- Changing the commission setting does not retroactively alter historical orders or already-created vendor payables.
- Super Admin can see and audit commission-rate changes.
- Financial screens show gross sales, commission rate, commission amount and vendor payable clearly.
 - Customer cannot order without authentication.
 - Customer can order from one or multiple restaurants.
 - Pickup sequence and walking-time scheduling work correctly.
 - Capacity/cutoff/hours rules prevent impossible pickup promises.
 - Razorpay payment state is server-authoritative and idempotent.
 - Each restaurant order has a unique single-use QR.
 - Only authorized restaurant staff/vendor can scan and collect.
 - Vendor Admin cannot see other restaurants.
 - Staff cannot access vendor/admin/customer-private functions outside operational scope.
 - Vendor can manage only permitted product fields.
 - Vendor has maximum 5 active staff in V1.
 - Vendor payout is manual and auditable.
 - Customer grievances always route to UNI8 Super Admin/support.
 - Vendor grievances always route to UNI8 Super Admin/support.
 - Super Admin can manage every supported restaurant, vendor admin, staff, product, order, payment and configuration.
 - Super Admin can reset/disable/force-logout staff/vendor accounts without seeing plaintext passwords.
 - Customer 360 exposes the complete authorized operational history of a customer.
 - Grievance CRM supports assignment, SLA, chat, internal notes, attachments, resolution and reopen.
 - Audit log captures privileged actions.
 - Financial records are traceable and not silently deleted.
 - Security phase has no unresolved critical/high issues.
 - Final debugging phase has no unresolved critical/high bugs.
 - Final production build and end-to-end regression pass.

24. Developer Changeover Rules

- Developer 1 owns Phases 1–3 only.
- Developer 2 receives Developer 1's mandatory handover and owns Phases 4–6.
- Developer 3 receives Developer 2's mandatory handover and owns Phases 7–9.
- Developer 4 receives Developer 3's mandatory handover and owns Phases 10–11.
- No developer is permitted to rely on undocumented private knowledge as a project dependency.
- Every developer must preserve the existing working features unless a documented change is required by this SRS.
- Every developer must update documentation when changing architecture, database, permissions, APIs, environment variables or external integrations.
- Every handover ZIP must be versioned and dated.
- Real secrets, production passwords, API secrets or private keys must NEVER be included in the ZIP or handover documents.
- Incoming developer must first restore/run the handed-over system and verify the documented baseline before beginning new work.
- Any deviation from this SRS must be documented with reason, affected modules and approval before being treated as the new requirement.

25. Official Technical Reference Notes

The following current official documentation should be re-checked by the implementing developer during the relevant phase because provider APIs, security guidance and limits can change.

Supabase Database: <https://supabase.com/docs/guides/database/overview>

Supabase RLS: <https://supabase.com/docs/guides/database/postgres/row-level-security>

Supabase Next.js Auth: <https://supabase.com/docs/guides/auth/quickstarts/nextjs>

Supabase Connection Management: <https://supabase.com/docs/guides/database/connecting-to-postgres>

Razorpay Webhooks: <https://razorpay.com/docs/webhooks/>

Razorpay Payments: <https://razorpay.com/docs/payments/>

Final Implementation Control Matrix

The following matrix is a final cross-check for the development teams. A phase is not complete merely because its screens exist; the underlying permissions, data integrity, financial correctness and handover documentation must also be complete.

Customer	Authentication required; valid profile; multi-restaurant cart; pickup scheduling; Razorpay payment; QR per restaurant; order history; rating; central grievance access.
Vendor Admin	Restaurant-scoped access; dashboard; analytics; products; orders; scanner; staff; payment visibility; vendor-to-UNI8 grievance.
Staff	Exactly Orders + Scan; restaurant scoped; max 5 active staff; secure access; QR collection only for assigned restaurant.
Super Admin	Global control; restaurant workspaces; Customer 360; central grievance CRM; access control; payments/disbursements; settings; analytics; audit.
Finance	8% launch commission is configurable, not hardcoded; vendor payable is 92% at launch; order-level commission snapshot; manual disbursement; proof; acknowledgement; audit trail.
Security	RLS, RBAC, secure sessions, no plaintext passwords, secret protection, payment verification, webhook idempotency, QR anti-replay, private files, rate limiting and security remediation.
Quality	End-to-end regression; load/performance checks; final production build; no unresolved critical/high bugs; final documentation and restore/deploy package.

Release rule

Phase 10 and Phase 11 are release-blocking. The project does not proceed to production merely because all UI/features are complete. Security vulnerabilities and critical/high bugs must be fixed and re-tested.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2 MASTER ADDENDUM — ORIGINAL V1 PRESERVED IN FULL

Campus Food Ordering • Scheduled Pickup • Vendor Operations • Super Admin Control

Version: V2 Amendment / Master Consolidation

Date: 25 August 2026

Purpose: Add the product-owner-confirmed decisions and additional V1 features to the original 25-page SRS without deleting any original requirement.

Master-document rule: The original V1 SRS appears unchanged before this addendum. This addendum is binding. Where it explicitly resolves an ambiguity, the V2 decision governs that ambiguity; all other original V1 requirements remain active.

A. OWNER-CONFIRMED V1 DECISIONS

- Customer cancellation is prohibited once payment is successful and the order is placed.
- A restaurant cancellation creates a penalty equal to 49% of the cancelled order value. The penalty is an auditable financial obligation and must not be hardcoded in frontend logic.
- UNI8 commission is 8% of the final amount paid by the customer for the applicable vendor order.
- V1 has no platform fee and no discount/coupon infrastructure.
- Customer refunds are manually initiated by Super Admin after a grievance/support review.
- A multi-restaurant purchase uses one customer-facing QR. Each restaurant's scanner resolves the same QR but exposes and authorizes only that restaurant's own order/items.
- Automated payment reconciliation is not a V1 requirement; Super Admin handles payment/payout issues manually.
- Quantity-based inventory is optional: a Vendor Admin may enable/use it if the restaurant wants it; boolean availability remains valid.
- V1 customer/vendor operational notifications use SMS. The developer must select the best India SMS provider based on DLT/TRAI compliance, delivery reliability, API quality, pricing, support and scalability, and must integrate it behind a provider abstraction.
- All original features remain; the additions below are additive.

B. ORDER PREPARATION & AUTOMATIC READINESS

B.1 Authorized preparation actions

- Staff may start preparation and mark an order ready for pickup for their assigned restaurant.
- Vendor Admin may start preparation and mark an order ready for pickup for their assigned restaurant.
- Super Admin retains platform-wide operational correction/control as already specified.
- Customers cannot change preparation/readiness state.

B.2 Automatic readiness rule

- If the scheduled pickup time is T, an order that has not already been manually marked ready becomes ready_for_pickup automatically at T + 5 minutes.
- Example: pickup time 1:15 PM → automatic ready-for-pickup at 1:20 PM.
- A Staff member or Vendor Admin may mark the order ready before T+5.
- The automatic transition is server-side, timezone-aware and idempotent.
- Automatic readiness must not overwrite a valid earlier manual ready state.
- Manual and automatic transitions must appear in the order timeline/audit history.
- Super Admin live operations must highlight orders that are still not ready after the expected readiness point.

B.3 Operational status visibility

- Restaurant users see pickup time, preparation state, ready state and overdue indicators.
- Customers see a simple pickup status/countdown; the countdown is informational and server state remains authoritative.

C. CANCELLATION, PENALTY & REFUND

C.1 Customer cancellation

- After successful payment verification and order placement, the customer cannot cancel.
- No Cancel Order action is shown for paid/placed orders.
- If the customer wants money returned because of an operational issue, the customer is directed to the UNI8 grievance/support flow.

C.2 Restaurant cancellation

- An authorized Vendor Admin or Super Admin may cancel a restaurant order under the platform's operational rules.
- Restaurant cancellation requires a mandatory reason.
- A cancellation creates an immutable cancellation event with actor, restaurant, order, timestamp, reason, penalty rate and penalty amount.
- The penalty is 49% of the cancelled order value. For V1 financial consistency, the cancelled order value is the final amount paid by the customer for that restaurant order.
- The penalty must be database-backed/configurable and auditable; it must not be a hardcoded frontend constant.
- The penalty must be visible in the restaurant financial view, order financial view and audit trail.
- The penalty is a separate financial obligation/event and must not overwrite the original sale or commission record.

C.3 Refund workflow

- Customers raise a grievance from the order when a refund is needed.
- Super Admin reviews the grievance and manually initiates the refund.
- V1 does not automatically refund customers.
- Refund events must record request, decision, initiation, success/failure, actor, timestamp and linked payment/order.

- For multi-restaurant purchases, Super Admin can process the affected restaurant order(s) without automatically refunding unrelated restaurant orders.
- Financial corrections remain reversal/adjustment events, consistent with the original SRS.

D. FINANCIAL MODEL — V2 CLARIFICATION

D.1 Commission

- UNI8 commission = $8\% \times \text{final amount paid by the customer for the applicable vendor order}$.
- Vendor payable at launch = final customer-paid amount – UNI8 commission, before separately approved financial adjustments.
- Example: ■10,000 final customer-paid amount → ■800 commission → ■9,200 vendor payable.
- No platform fee exists in V1.
- No discount/coupon/promotion engine exists in V1.
- Historical commission rate snapshots remain mandatory.

D.2 Manual vendor payouts

- Vendor payouts remain manual and are executed by Super Admin.
- The system must prevent duplicate payout records for the same covered payable unless an explicit audited reversal/adjustment exists.
- Every payout records amount, restaurant/vendor, covered payable, reference, actor, timestamp, proof and acknowledgement.
- Restaurant cancellation penalties must be visible as separate ledger events and may be included in settlement calculations only through an explicit audited financial action.

D.3 Payment/reconciliation scope

- A dedicated automated payment reconciliation engine is not required in V1.
- Super Admin remains responsible for manually resolving payment/payout discrepancies using the existing financial ledger, grievance workflow and new reconciliation dashboard.

E. SMS NOTIFICATIONS & OTP

E.1 Provider selection

- Developer must evaluate and select the best India-capable SMS provider for UNI8 rather than embedding a provider assumption into business logic.
- Selection criteria: TRAI/DLT compliance, sender/header/template support, India operator reach, delivery reliability, OTP support, transactional SMS support, API/webhook quality, latency, pricing, support, scalability, monitoring and provider portability.
- The provider integration must be behind an internal SMS/notification service interface so UNI8 can switch providers without changing order business logic.
- The implementation team must re-check current DLT/provider requirements immediately before launch. Current India SMS provider documentation confirms that DLT registration, sender/header and approved templates are required for compliant Indian messaging.

E.2 V1 SMS events

- Phone OTP/login.
- Order placed/payment confirmed.
- Pickup reminder.
- Order ready for pickup.
- Restaurant cancellation.
- Refund initiated/completed where applicable.
- Grievance update/reply.

- Super Admin-enabled operational announcements.

E.3 SMS reliability/security

- SMS sending is server-side.
- OTP requests and resend actions are rate-limited.
- Provider message IDs and delivery/failure states are stored where supported.
- Provider credentials/secrets are never exposed to the browser.
- The architecture should permit a future fallback provider.

F. SUPER ADMIN LIVE OPERATIONS COMMAND CENTER

This is additive to the original Super Admin Dashboard and Analytics; neither is removed.

- Orders due for pickup soon.
- Orders not started/preparing.
- Orders not ready after expected readiness.
- Orders past pickup time.
- Restaurants currently paused.
- Restaurants nearing/exceeding pickup capacity.
- Payment/order exceptions requiring manual attention.
- QR scan failures and suspicious scan activity.
- Urgent/high grievances and SLA breaches.
- Vendor payouts awaiting acknowledgement.
- Restaurant cancellations and 49% penalty events.

F.1 Behaviour

- Every alert links to the underlying order, restaurant, grievance, payment, QR or payout record.
- Live data is server-authoritative.
- V1 does not require permanent Realtime subscriptions for every record; periodic refresh/server aggregation is acceptable.
- Operational alerts must be auditable when acknowledged/resolved.

G. RESTAURANT TEMPORARY PAUSE ORDERING

- Vendor Admin and Super Admin can temporarily pause new orders for a restaurant.
- Pause may use a selected duration or manual reopen.
- Examples: 15 minutes, 30 minutes, 1 hour, or until manually reopened.
- Pausing blocks new checkout/order placement but does not delete or automatically cancel existing paid orders.
- Existing paid orders remain visible and continue through their operational lifecycle unless separately cancelled by an authorized restaurant/Super Admin action.
- Customers see a clear temporary-unavailable message.
- Pause/reopen actions are audited.

H. CUSTOMER PICKUP COUNTDOWN & ORDER EXPERIENCE

- Customer order detail shows scheduled pickup time and current order state.
- Customer may see statuses such as Pickup in X minutes, Preparing, Ready for pickup, Pickup window ending soon and Pickup window expired.
- The countdown must use server time/authoritative timestamps rather than trusting the browser clock.
- Each restaurant order inside a multi-restaurant group has its own pickup time and status.

I. CUSTOMER ORDER-ISSUE SHORTCUTS

- Customers do not access the internal CRM.
- A Need Help / Report an Issue action is available from the relevant order.

- Issue choices: Wrong item, Missing item, Food/order issue, Pickup issue, QR problem, Payment/refund issue, Restaurant issue and Other.
- The selected issue automatically pre-populates order ID, restaurant, customer and category before opening the UNI8 grievance flow.
- All such tickets are handled by UNI8 Super Admin/support and never routed to Vendor Admin.

J. MULTI-RESTAURANT ORDER GROUP & UNIFIED QR

- A multi-restaurant checkout creates one customer-facing order group and one QR.
- The customer view clearly lists each restaurant, each restaurant order, pickup time, status and collection state.
- The same QR can be scanned at Restaurant A and later at Restaurant B.
- When Restaurant A scans the QR, the server resolves the group but returns only Restaurant A's own order/items/pickup information.
- Restaurant A cannot view Restaurant B's items, order details, financial information or private customer data.
- After Restaurant A collects its order, the QR remains usable for the remaining eligible restaurant orders.
- Collection is atomic per restaurant order; a successful scan at one restaurant does not mark other restaurant orders collected.
- A restaurant cannot use the QR to collect an order belonging to another restaurant.
- The unified QR must preserve the original SRS's anti-replay, payment, status, identity and restaurant-scope checks.

K. QR FALLBACK VERIFICATION

- V1 includes a controlled fallback for genuine QR scanning failures.
- The fallback credential must be short-lived, non-guessable and tied to the relevant order/group/customer context.
- Fallback requires authenticated Staff/Vendor Admin identity and restaurant scope.
- Fallback cannot bypass payment verification, order state, restaurant scope or collection checks.
- All fallback attempts are logged.

L. CHECKOUT REVALIDATION & STALE CART PROTECTION

- Immediately before payment/order creation, the server revalidates restaurant status, product availability, current price, operating hours, pickup slot, capacity, preparation cutoff and schedule feasibility.
- If a restaurant pauses/closes or a product/slot becomes unavailable after being added to the cart, checkout stops and explains what changed.
- Customer can remove the affected selection or choose a newly valid pickup option.
- Browser cart values are never trusted for final payment/order truth.

M. OPTIONAL QUANTITY INVENTORY

- Vendor Admin may choose to manage a product using quantity-based stock if desired.
- Quantity inventory is optional; a restaurant may continue using simple available/out-of-stock status.
- If quantity mode is enabled, stock decrements/reservations must be server-side and protected against race conditions.
- A product reaching zero quantity becomes unavailable until replenished/re-enabled according to the restaurant's configuration.

N. FUTURE-READY PROMOTION ARCHITECTURE

- V1 does not activate coupons, discounts or promotions.
- The data architecture may reserve future promotion/promotion-redemption entities so future versions do not require a destructive redesign.
- No promotion may affect V1 pricing or commission unless a future approved SRS explicitly enables it.

O. SUPER ADMIN ANNOUNCEMENTS

- Super Admin can create platform/customer announcements.
- Announcement fields may include title, message, severity, target scope, start time, end time and active/published state.
- Announcements are visible to customers only when the Super Admin explicitly turns/publishes them on.
- Announcements may be global or restaurant-specific where supported.
- Create/edit/publish/unpublish/archive actions are audited.

P. DATA GOVERNANCE & RETENTION

- Define retention/archival rules for customer profiles, orders, payments, grievances, ratings, audit logs, payout proofs, attachments and authentication/security events.
- Historical financial and order records must remain auditable as required by the original SRS.
- Use deactivation, archival or controlled anonymization where destructive deletion would break financial/audit integrity.
- Private grievance attachments and payout proofs remain access-controlled throughout retention.
- Retention periods and deletion/anonymization behaviour must be documented before production launch and aligned with applicable legal/provider requirements.

Q. FEATURE FLAGS

- Super Admin Platform Settings supports database-backed feature flags.
- Feature flags must be enforced server-side for any feature that changes security, financial, ordering or access behaviour.
- Examples include multi-restaurant ordering, ratings, announcements, optional inventory, new checkout variants and future promotions.
- Changing a feature flag is audited.
- A disabled feature must be blocked at server/action level, not merely hidden in the UI.

R. MAINTENANCE MODE

- Super Admin can enable maintenance mode globally or for supported modules/order entry.
- Existing paid orders remain accessible and manageable unless a separate emergency procedure is invoked.
- Customers receive a clear maintenance message rather than a generic failure.
- Maintenance activation/deactivation is audited and server-enforced.

S. FRAUD & ABUSE DETECTION

- The platform records operational fraud/abuse flags without automatically banning a user solely from a heuristic.
- Customer signals may include excessive OTP requests, repeated failed payment attempts, repeated no-shows or abnormal account behaviour.
- QR signals may include repeated failed scans, cross-restaurant scan attempts, repeated scans after collection or abnormal scan frequency.
- Vendor signals may include unusual cancellation rates, manual adjustments, payout anomalies or abnormal operational patterns.
- Flags are data-driven, auditable and reviewable by Super Admin.
- Super Admin can acknowledge, investigate and resolve flags.

T. FINANCIAL RECONCILIATION DASHBOARD

- Add Finance → Reconciliation to the Super Admin area.
- Compare Razorpay payment records, UNI8 orders, commission records, vendor payables, manual disbursements and refund/adjustment events.
- Highlight payment-without-order, order/payment mismatch, duplicate payment event, refund mismatch, duplicate payout and payable mismatch.
- Each mismatch links to the underlying records.
- Resolution is manual and auditable in V1; the dashboard does not introduce automated vendor payouts or automated refunds.

U. ADDITIONAL DATABASE / DOMAIN REQUIREMENTS

Entity / extension	Purpose
restaurant_cancellation_events	Immutable cancellation reason, actor, penalty rate/amount and financial impact.
optional_product_inventory	Optional quantity-based stock and inventory state.
notifications	Notification records and delivery state.
sms_provider_events	Provider message ID, delivery/failure state and template metadata.
feature_flags	Server-enforced platform feature configuration.
announcements	Super Admin customer-facing operational notices.
maintenance_mode	Global/module maintenance state.
fraud_flags	Operational abuse/security flags and resolution history.
financial_reconciliation_items	Financial mismatches and audited resolution records.

V. DEVELOPMENT PHASE INTEGRATION

The original 11 phases remain unchanged. The following deliverables are added to those phases.

Developer 1 — Phases 1–3

- Phase 2: restaurant pause state, stale-cart revalidation, pickup countdown model, multi-restaurant order-group UX and unified QR model.
- Phase 3: SMS abstraction, OTP/transactional SMS foundation, unified QR authorization and QR fallback verification.

Developer 2 — Phases 4–6

- Phase 4: Vendor Admin preparation controls, optional quantity inventory, pause ordering and restaurant cancellation.

- Phase 5: Staff preparation controls and automatic T+5 readiness.
- Phase 6: 49% restaurant cancellation penalty ledger, manual refund support and payout duplicate protection.

Developer 3 — Phases 7–9

- Phase 7: Live Operations Command Center and enhanced restaurant operations.
- Phase 8: Customer-facing order issue shortcuts feeding the central grievance CRM; customer does not access the CRM.
- Phase 9: SMS administration, announcements, feature flags, maintenance mode, fraud/abuse flags and reconciliation dashboard.

Developer 4 — Phases 10–11

- Security audit must cover SMS provider secrets/DLT configuration, unified QR cross-restaurant isolation, fallback verification, feature-flag enforcement, maintenance-mode authorization and abuse controls.
- Full debugging must test automatic T+5 readiness, manual-vs-automatic readiness races, restaurant cancellation penalties, stale carts, unified QR scans, SMS retries and reconciliation edge cases.

W. V2 ACCEPTANCE CRITERIA

- Staff and Vendor Admin can start preparation and mark orders ready.
- A 1:15 PM pickup automatically becomes ready at 1:20 PM if not already marked ready.
- Customer cannot cancel after successful payment and placement.
- Restaurant cancellation creates a 49% penalty event and audit trail.
- 8% commission is calculated on the final amount paid by the customer; no V1 platform fee or discount engine exists.
- Customer refund requests go through UNI8 grievance/support and are manually initiated by Super Admin.
- One QR works across a multi-restaurant order group while each restaurant can only see/collect its own order.
- QR fallback is secure, short-lived, restaurant-scoped and audited.
- Customer can report an order issue directly from the order and the ticket reaches UNI8 support.
- SMS provider is selected using documented India/DLT/delivery/API/cost/support/scalability criteria.
- Required transactional/OTP SMS events work through the selected provider.
- Super Admin has a live operational command center in addition to the original dashboard/analytics.
- Super Admin/Vendor Admin can temporarily pause restaurant ordering without deleting existing paid orders.
- Customer sees pickup countdown/status information.
- Multi-restaurant order groups clearly show each restaurant's pickup time and status.
- Checkout revalidates restaurant/product/slot/capacity state immediately before payment/order creation.
- Promotions remain disabled in V1 while architecture may be future-ready.
- Customer announcements appear only when Super Admin explicitly enables/publishes them.
- Feature flags are server-enforced and audited.
- Maintenance mode is server-enforced and audited.
- Fraud/abuse flags are reviewable and auditable.
- Super Admin can compare payments, orders, payables, disbursements and refunds/adjustments through reconciliation.

X. V1 NON-REQUIREMENTS CONFIRMED

- Automated vendor payouts remain out of V1.
- Automated customer refunds remain out of V1.
- Menu time-of-day scheduling remains out of V1.
- Disaster-recovery RPO/RTO targets are not being added as a V1 feature requirement.
- Original exclusions remain excluded: delivery, native mobile apps, loyalty/rewards, referrals, AI recommendations, public social feed, live map tracking, cash payments and vendor self-service platform settings, unless a future approved SRS revision changes them.

Y. SMS IMPLEMENTATION NOTE

The implementation team must re-check provider documentation immediately before production because India messaging/DLT rules and provider APIs can change. MSG91 is a viable current India-focused baseline with official DLT and OTP documentation, but UNI8 is not locked to that provider; the developer must document the final provider selection and rationale before production.

Z. FINAL DEVELOPER INSTRUCTION

The original 25-page V1 SRS plus this V2 addendum together form the UNI8 master development specification. Developers must implement all original requirements and all applicable additions. UI completion alone does not constitute

completion: authorization, server-side enforcement, data integrity, auditability, financial correctness, race-condition handling, failure states, security and end-to-end acceptance must pass. The original Phase 10 and Phase 11 release-blocking rules remain unchanged.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.1 Frontend Brand Identity & UI Experience Addendum

This addendum is part of the global UNI8 SRS and supplements the existing V2 Master SRS. It does not remove, replace, or weaken any previously defined functional, security, financial, role, operational, architectural, testing, or release requirement.

26. Brand Assets & Frontend Design Direction

UNI8's customer-facing frontend must be designed as a distinctive campus-food brand experience, not as a generic food-ordering dashboard. The supplied UNI8 brand-asset ZIP is the authoritative visual source for the frontend brand identity. The package may contain the UNI8 primary logo, stand-alone logo mark, mascot, logo-with-mascot composition, and other supplied brand assets. All supplied assets must be inspected before interface generation.

26.1 Mandatory Brand-Asset Inspection

- **Inspect every supplied brand asset carefully before generating interfaces.** The developer/design system must not begin from a generic UI kit and then attach the UNI8 logo afterward.
- Extract and document the exact visual language represented by the assets, including colour relationships, visual hierarchy, logo proportions, mascot personality, typography character, graphic treatment, spacing character, shape language, contrast, and overall brand feel.
- The resulting interface must look as though it was designed by the same brand designer who created the supplied UNI8 assets.
- The logo, stand-alone logo, mascot, and logo-with-mascot variants must be used according to their intended visual roles. Do not distort, stretch, recolour, redraw, or alter supplied marks unless the brand assets explicitly provide that variant.
- Do not introduce unrelated colours, illustration styles, typography personalities, icon treatments, gradients, shadows, corner radii, or decorative motifs merely because they are common in generic SaaS/food-delivery templates.
- Where the supplied asset package and this SRS appear to conflict on visual identity, the supplied brand assets control the visual identity while this SRS controls product functionality and behaviour.

26.2 UNI8 Customer Frontend Experience

The customer website should take broad experiential inspiration from the energy and usability of leading food-ordering platforms such as Zomato and Swiggy, while remaining unmistakably UNI8. The objective is not to clone either product's interface, layout, copy, illustrations, or visual identity.

- Fun, playful and energetic rather than corporate or sterile.
- Strong college/campus vibe appropriate for university students.
- Fast, visual and highly scannable discovery of restaurants, products and pickup availability.
- Large, expressive food imagery and clear product hierarchy where supported by the supplied brand identity.
- Micro-interactions, motion and playful feedback may be used where they improve the experience without slowing ordering.
- The interface should feel social and youthful through composition, copy tone, motion and visual rhythm, while remaining professional enough for a real university commerce platform.
- Ordering, pickup timing, QR pickup and support actions must remain extremely obvious even when the visual design is playful.
- Mobile-first responsive behaviour is required even though V1 is a website; the desktop experience must remain polished and functional.

26.3 Design-System Requirements

Area	Requirement
Colour	Build the UI colour system from the supplied UNI8 brand assets. Define primary, secondary, accent, surface, background, text, success
Typography	Extract the character of the supplied brand identity and select production-ready web fonts that preserve that character. Typography mus
Logo	Use the correct UNI8 logo variant by context. Maintain proportions, clear space and legibility.
Mascot	Use the mascot as a deliberate brand character, not as decoration pasted randomly into screens. Define appropriate placements, expre
Components	Buttons, cards, chips, tabs, inputs, menus, modals, empty states, alerts and status indicators must belong to one coherent UNI8 design
Motion	Use subtle purposeful transitions and feedback for actions such as adding to cart, payment success, pickup readiness, QR presentation
Responsive	Customer ordering must work cleanly on mobile, tablet and desktop widths. Touch targets and sticky checkout/pickup actions must be c
Accessibility	Playfulness must not compromise readability, keyboard navigation, focus states, semantic structure, contrast, or usable error messaging

26.4 Interface Generation Control Prompt

The following instruction is mandatory for any AI-assisted UI generation, design generation, frontend scaffolding, or design-to-code workflow used for UNI8. It is included in the SRS so that developers do not accidentally generate a generic food-delivery interface and apply the logo afterward.

Before generating the interfaces, inspect all five supplied UNI8 brand assets carefully. Extract the exact visual language, colour relationships, logo proportions, mascot personality, typography character, and overall brand feel from them. The resulting UI should look like it was designed by the same brand designer who created these assets. Do not merely paste the logo into a generic template.

Build the UNI8 frontend as a fun, playful, energetic campus-food experience with the usability and visual immediacy expected from leading food-ordering platforms such as Zomato and Swiggy, but do not copy their branding, layouts, illustrations, text, or distinctive UI patterns. UNI8 must have its own unmistakable visual identity derived from the supplied brand assets.

Use the supplied logo, stand-alone logo, mascot, logo-with-mascot composition and all other supplied assets intentionally and consistently. Preserve the proportions and character of the original assets. Establish a coherent UNI8 design system for colours, typography, spacing, cards, buttons, icons, status states, illustrations, motion, empty states, loading states and error states.

The customer experience should feel youthful, college-oriented and food-first: visual, fast, expressive and playful without becoming childish or confusing. Ordering, pickup timing, QR pickup and support must remain immediately understandable. Prioritize excellent mobile responsiveness while maintaining a polished desktop experience.

Do not invent a separate visual identity. Do not use a generic SaaS dashboard aesthetic for the customer experience. Do not introduce arbitrary colours or visual motifs that conflict with the supplied brand assets. The final result must feel like one coherent UNI8 brand ecosystem rather than a collection of screens.

26.5 Required Frontend Review Before Phase Completion

- Developer/design owner must provide a visual review showing the major customer screens using the supplied assets.
- The review must confirm that the logo and mascot are being used consistently and that the interface does not resemble a generic food-delivery template.
- The review must include responsive views for key customer journeys: discovery, restaurant/menu, product, cart, multi-restaurant checkout, payment, order tracking, QR pickup and grievance creation.
- The review must verify loading, empty, error, success, unavailable, out-of-stock and operational-alert states.
- Any AI-generated frontend code or design must be manually reviewed against the supplied brand assets before acceptance.
- The brand asset ZIP must be preserved as part of the project asset inventory and handover package; developers must not replace the authoritative originals with compressed screenshots or re-created approximations.

26.6 Scope Boundary

This requirement changes the presentation and design quality of the frontend; it does not change the existing UNI8 functional requirements, role permissions, financial model, grievance ownership, payment architecture, QR rules, security requirements, development phases, or V1 exclusions. All previous SRS requirements remain in force.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.2 Phone-First, University Access & Navigation Addendum

This addendum supplements the complete UNI8 V2.1 Master SRS. No existing requirement is removed unless this addendum explicitly changes it. All security, finance, payment, grievance, QR, RBAC, audit, handover, testing and release requirements remain active.

27. Phone-First Product Design Requirement

UNI8 V1 shall be designed and implemented as a phone-first web product. The primary design target for the customer frontend AND all authenticated dashboards is a modern smartphone viewport. Desktop and tablet layouts are secondary responsive adaptations, not the primary design source.

- Customer screens must be designed first for mobile-width interaction and then expanded responsively.
- Vendor Admin, Staff and Super Admin dashboards must also use a phone-first responsive system. Desktop may expose additional density, columns and panels, but the mobile information hierarchy remains the foundation.
- All primary actions must be accessible without horizontal scrolling.
- Large dashboard tables must have mobile cards, stacked rows, expandable details or controlled scrolling where genuinely necessary.
- Dialogs, drawers, bottom sheets and action menus must be touch-first.
- Touch targets, spacing, sticky actions and safe-area behaviour must be considered.
- Common smartphone viewports must be tested before desktop polish is considered complete.
- Phone-first does not mean mobile-only; tablet and desktop remain fully supported.

28. Customer Main Navigation

The customer mobile website shall use persistent bottom navigation with exactly four primary destinations:

Position	Destination	Purpose
1	Home	Current/personalized discovery and active ordering context.
2	Explore	Restaurant and product discovery, search and filtering.
3	Orders	All current, upcoming and historical orders.
4	Profile	Profile, account/security, ratings and support access.

- Bottom navigation is persistent on the primary customer mobile experience except focused flows such as payment or full-screen QR presentation.
- The active destination must have a clear UNI8-branded state.
- No emoji may be used as navigation icons or labels.
- Desktop may adapt this into a suitable header/sidebar pattern, while these four destinations remain canonical.

29. University-Internal Restaurant Access

Every restaurant must be explicitly classified by Super Admin as Inside University or Outside University. This classification controls the customer-facing access warning for internal restaurants.

29.1 Super Admin Configuration

- When creating or editing a restaurant, Super Admin must see a required Restaurant Location Type dropdown.

- Allowed values: Inside University and Outside University.
- If Inside University is selected, Super Admin must configure a University Place Name, such as the campus building, food court or other access-controlled place.
- The place name is database-backed and must never be hardcoded in the frontend.
- Location type and place-name changes are audit logged.
- Relevant restaurant configuration must remain auditable with historical orders.

29.2 Customer Popup

When a customer selects an Inside-University restaurant, a mandatory access-awareness popup must appear before the customer proceeds into that restaurant's ordering experience.

- Required message concept: "You can only order from [PLACE NAME] if you are a valid student/faculty/staff member who can enter the university."
- The area outside the popup must be dimmed, not blurred.
- The popup contains an OK action.
- OK displays a five-second countdown in brackets: OK (5), OK (4), etc., ending as OK when the countdown reaches zero.
- The popup cannot be closed, dismissed or bypassed before the five-second countdown completes.
- The countdown is a UX/access-awareness control, not proof of university eligibility.
- After the countdown, selecting OK permits continuation.
- If the customer leaves before confirming, the acknowledgement is not recorded.
- The popup must work cleanly on phone-first and desktop layouts.

29.3 Product Boundary

V1 does not define university SSO, student-ID verification or access-control integration. The popup communicates the physical access restriction and places responsibility on the customer to be a person permitted to enter. Any future automated eligibility verification is a separate requirement.

30. Global No-Emoji Frontend Policy

UNI8 V1 must not use emojis anywhere in the website UI. This applies to every customer page, Vendor Admin dashboard, Staff portal, Super Admin dashboard, popup, modal, alert, notification component, empty state, loading state, error state, button, menu, status display and other frontend interface.

- Do not use Unicode emoji as icons.
- Do not use decorative emoji in headings, labels, empty states or in-application marketing copy.
- Do not use emoji in dashboard cards or operational alerts.
- Use approved UNI8 iconography, typography, shapes, illustrations and mascot assets instead.
- The supplied mascot is a brand asset and may be used according to the V2.1 brand requirements.
- The prohibition applies equally to manually coded and AI-generated frontend.

31. Zomato / Swiggy-Inspired Customer Experience

The main customer frontend should take product-experience inspiration from Zomato and Swiggy mobile apps: fast discovery, strong food imagery, prominent search, rich restaurant cards, easy cart access, clear pricing and polished mobile ordering. This is an experience benchmark, not a request to copy their branding, proprietary layouts, illustrations or distinctive visual identity.

- UNI8 supplied brand assets remain authoritative for colour, typography, logo, mascot and brand personality.
- The frontend should feel fun, playful, youthful and campus-oriented while remaining trustworthy for commerce.
- Restaurant availability, pickup time, preparation status, payment status and QR pickup must be immediately understandable.
- Prefer visual discovery and short interaction paths over dense enterprise-style presentation.
- The four-tab mobile navigation in Section 28 is mandatory.

32. Development Allocation & Phase Rebalancing

These additions must not be dumped wholesale onto Developer 1. Developer 1 has already started Phase 1–3 work. Existing progress must be preserved; the new requirements are introduced through controlled incremental changes and subsequent phase ownership.

Requirement	Ownership / handling
Phone-first design system	Cross-project standard. Do not restart Developer 1's completed work. Incoming developers apply it to their modules and
Customer bottom navigation	If Developer 1's customer frontend is already substantially implemented, preserve it and carry the navigation change thr
University restaurant classification	Developer 3 / Super Admin restaurant-management scope, coordinated with the next applicable database migration ow
University access popup	May be added to the current customer module owner as an incremental requirement; it does not justify a wholesale Pha
No-emoji policy	Global design-system and QA requirement. Every developer follows it; Developer 4 verifies it during final review.
Zomato/Swiggy-inspired UX	Cross-project design direction. Developer 2/3 apply it to their modules; existing Developer 1 work is refined through con

Developer 1 remains responsible for the existing Phase 1–3 work already underway and is not required to rebuild completed functionality solely because of this addendum. Any change affecting Developer 1-owned modules must be documented in the handover/change report and implemented as controlled incremental work.

33. Updated Acceptance Criteria

- Customer frontend and all dashboards are phone-first and responsive.
- Customer mobile navigation exposes Home, Explore, Orders and Profile.
- Super Admin can classify each restaurant as Inside University or Outside University.
- Inside-University restaurants have a configurable university place name.
- Selecting an Inside-University restaurant triggers the mandatory popup.
- Popup background is dimmed and never blurred.
- OK remains unavailable for dismissal until its five-second countdown completes.
- Popup does not falsely claim automated verification of student/faculty/staff status.
- No emoji appears anywhere in the website frontend or dashboards.
- Customer UX is inspired by the speed and discovery patterns of Zomato/Swiggy while remaining distinctly UNI8.
- Supplied UNI8 brand assets remain the authoritative visual identity.
- These additions do not require Developer 1 to restart Phase 1–3.

34. Change-Control Rule

This V2.2 addendum is cumulative. The original SRS, V2 additions and V2.1 brand/frontend requirements remain active. Where V2.2 specifically refines UI behaviour, V2.2 controls that behaviour; otherwise all previous requirements remain unchanged.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.3 Supabase Environment, Developer Access & Handover Addendum

This addendum supplements the complete UNI8 V2.2 Master SRS. It does not remove existing product, security, financial, operational, frontend, database, handover or release requirements.

35. Supabase Developer Access Structure

UNI8 shall use a controlled Supabase access model tied to the four-developer handover structure. **Developer 1 must NOT receive direct access to the UNI8 Supabase project.** Supabase project setup and direct project access begin after Developer 1 completes Phases 1–3.

35.1 Developer 1 — No Supabase Project Access

Developer 1 may design and implement the required database schema, migrations, SQL, RLS policies, functions, triggers, seed data and Supabase integration structure as code/documentation, but must not receive direct access to the actual UNI8 Supabase project.

- Developer 1 must not receive the Supabase Dashboard, project credentials, database password, service-role key, production secret or Supabase project access.
- Developer 1 must produce all database and Supabase artifacts needed to reproduce the intended Phase 3 state.
- Developer 1 must document every Supabase feature/configuration that Developer 2 must create or configure.
- Developer 1 must never request, receive, store or embed the owner's Supabase credentials or secrets.

35.2 Mandatory Supabase Setup Guide from Developer 1

At completion of Phase 3, Developer 1 must provide the owner with a detailed step-by-step Supabase setup guide. The guide must allow the owner to create/configure the Supabase project without undocumented developer knowledge.

- Project creation and required settings.
- Authentication configuration, including required provider and callback/redirect configuration.
- Database extensions, schemas, tables, relationships, indexes, constraints and enums.
- All SQL migrations in exact application order.
- All RLS policies and affected tables.
- Database functions, triggers and scheduled jobs where applicable.
- Storage buckets, privacy settings, structure and access policies.
- Required environment variables and where each value is obtained; no real secrets.
- Auth configuration and role/membership setup.
- Seed data and initial configuration values.
- Local/staging connection configuration.
- Verification tests proving the created project matches Phase 3 application expectations.
- Troubleshooting and provider-specific configuration required by Developer 2.

35.3 Owner Creates and Controls the Supabase Project

After receiving Developer 1's Phase 3 handover and Supabase setup guide, the UNI8 owner creates the Supabase project and performs the initial setup. The owner remains the primary controller of the project.

- The owner creates the Supabase project using the owner's account.
- The owner enables appropriate account security/MFA where supported.

- The owner never shares the owner's Supabase account password with any developer.
- The owner creates/invites the appropriate account for Developer 2.
- Production secrets, service-role credentials and database passwords are never placed in the handover ZIP or ordinary chat.

35.4 Developer 2 — First Developer with Supabase Access

Developer 2 is the first developer who receives direct access to the UNI8 Supabase project. Developer 2 receives access only after the owner creates the project and grants the appropriate account access.

- Developer 2 first reads Developer 1's Supabase setup guide and handover.
- Developer 2 verifies the project before beginning Phase 4.
- Developer 2 applies the Phase 1–3 migrations, RLS policies, functions, triggers, indexes, Storage configuration and other documented setup required to reproduce the Phase 3 baseline.
- Developer 2 must not silently alter Developer 1's schema/security model; deviations must be documented.
- Developer 2 verifies application/database compatibility after setup.
- Developer 2 updates Supabase/database documentation for material changes in Phases 4–6.

35.5 Developer 3 and Developer 4

Developer 3 and Developer 4 also receive Supabase access because their assigned phases require database, platform, security and final debugging work.

- Developer 3 receives access after Developer 2's Phase 6 handover and continues using the existing UNI8 Supabase project.
- Developer 3 applies and verifies the database changes required for Phases 7–9 and maintains authoritative migration history.
- Developer 4 receives access after Developer 3's Phase 9 handover for Phases 10–11.
- Developer 4 audits Supabase configuration, RLS, authentication, Storage, database permissions, secrets exposure, policies, functions and triggers during security review.
- Developer 4 performs final database/security regression verification.

36. Supabase Access Matrix

Access	Dev 1	Dev 2	Dev 3	Dev 4	Owner
Supabase project access	NO	YES	YES	YES	YES
Create Supabase project	NO	NO	NO	NO	YES
Prepare migrations/RLS	YES, artifacts only	YES	YES	YES	YES/controlled
Apply initial migrations	NO	YES	YES if restore needed	YES if required	YES/controlled
Auth configuration	Document	YES	YES	YES/security	YES
Storage configuration	Document	YES	YES	YES/security	YES
Production secrets	NO	No direct sharing	No direct sharing	No direct sharing	Controlled
Owner credentials	NO	NO	NO	NO	YES
Production changes	NO	Controlled	Controlled	Security/debug only	YES
Access after responsibility	NO	Owner-controlled	Owner-controlled	Owner-controlled	YES

37. Supabase Handover Sequence

Step 1: Developer 1 completes Phases 1–3 and prepares all application/database artifacts without accessing the actual Supabase project.

Step 2: Developer 1 hands over the complete project plus the mandatory detailed Supabase Setup Guide.

Step 3: The UNI8 owner creates the Supabase project and performs the documented initial setup.

Step 4: The owner grants Developer 2 the required Supabase project access. Developer 2 is the first developer with direct Supabase access.

Step 5: Developer 2 applies all Phase 1–3 migrations, RLS policies, functions, triggers, indexes, Storage configuration and other required setup, then verifies the Phase 3 baseline.

Step 6: Developer 2 continues Phases 4–6 using version-controlled database changes.

Step 7: Developer 3 receives access to the same Supabase project after Developer 2's handover and continues from that state.

Step 8: Developer 4 receives access to the same project after Developer 3's handover for security and full debugging.

Step 9: The owner remains the project controller and may revoke/restrict developer access at any time.

38. Non-Negotiable Supabase Security Rules

- Developer 1 must never receive direct Supabase project access.
- No developer receives the owner's Supabase account credentials.
- Real production secrets are never included in developer ZIPs, handover documents or source repositories.
- Service-role keys are never exposed to browser/client code.
- All database migrations and RLS changes are version-controlled and included in applicable handovers.
- Developers must not make undocumented manual production database changes.
- Any emergency manual production change must be documented and reconciled into the migration/documentation system.
- Developer access is revoked or reviewed when project responsibility ends.
- Every incoming developer verifies the handed-over application/Supabase baseline before changing it.
- Supabase project ownership remains with the UNI8 owner; developer access is delegated access, not ownership.

39. Updated Handover Requirement

All existing handover requirements remain unchanged. In addition, Developer 1's Phase 3 handover must contain the detailed Supabase Setup Guide defined above. Every subsequent handover must include the current Supabase migration state, RLS state, Storage/Auth configuration, environment-variable inventory and known Supabase-specific issues.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.4 In-House QR, Pickup Lifecycle & Basic Product Behaviour Addendum

This addendum is cumulative and binding. It supplements the complete UNI8 V2.3 Master SRS and does not remove existing product, security, financial, operational, frontend, database, Supabase, handover or release requirements. Where this addendum explicitly refines an existing behaviour, this V2.4 requirement controls that behaviour.

44. In-House UNI8 QR System

UNI8 V1 shall use an entirely in-house QR system. QR generation, QR display, browser-based QR scanning, server-side QR validation and pickup collection must be implemented as part of UNI8. UNI8 must not depend on a third-party QR generation service, QR hosting service, QR validation SaaS or third-party QR scanning SaaS.

44.1 QR Generation

- The backend must generate a cryptographically strong, non-guessable pickup authorization token for the applicable pickup authorization.
- Tokens must not be sequential, predictable or based solely on an order ID.
- The QR must not expose unnecessary sensitive information such as phone numbers, payment credentials or private customer data.
- The authoritative QR/pickup authorization record and lifecycle state must be stored in the UNI8 database.
- The customer receives the QR through the UNI8 Order interface.
- The QR becomes invalid when its associated pickup authorization is collected or otherwise invalidated by an authoritative order-state rule.
- If a QR must be reissued, the previous authorization must be invalidated before the new one becomes valid, and the event must be audit logged.
- The frontend must never be trusted to create or authorize a valid pickup merely because it displays a QR.

44.2 Browser-Based QR Scanning

The Staff and Vendor Admin Scan interfaces shall scan QR codes directly through the device camera using the UNI8 website. A separate scanner application is not required.

- The browser requests camera permission through the normal device/browser permission flow.
- The Scan screen must provide clear states for camera permission request, ready, scanning, success, invalid QR, denied permission, unavailable camera and technical/network failure.
- Camera access must stop when the scan screen is exited or scanning is otherwise terminated.
- The scanner must prevent accidental repeated processing of the same QR in a single scanning session.
- The scanner only reads the token; it does not decide whether pickup is valid.
- The token is sent to the UNI8 backend for authoritative validation.
- Unsupported browsers/devices and denied camera access must have a clear recovery/fallback message.

44.3 Server-Side Validation and Atomic Collection

- The scanning user must be authenticated.
- The scanning user must be Staff or an authorized Vendor Admin.
- The scanning user must be authorized for the restaurant against which the pickup is being performed.
- The QR token must exist and be valid.
- The token must not already have been consumed/collected/invalidated.

- The associated payment state must satisfy the authoritative payment requirement.
- The order must not be cancelled, refunded, no-show or otherwise ineligible for collection.
- The restaurant performing the scan must match the restaurant order being collected.
- The order must be in a collection-eligible state.
- Successful collection must be atomic so simultaneous scans cannot produce two successful pickups.
- Every successful and failed scan must be audit logged with actor, restaurant, timestamp and outcome/reason.
- The backend is the only authority that can change the order to collected.

44.4 Multi-Restaurant QR

For a multi-restaurant customer checkout, the customer receives one overall pickup QR. The same QR may be presented at multiple restaurants, but each restaurant scan operates only on that restaurant's own order.

- Restaurant A scanning the QR must receive only Restaurant A's relevant order/pickup information.
- Restaurant B scanning the same QR must receive only Restaurant B's relevant order/pickup information.
- A restaurant must never gain access to another restaurant's private order details through the shared QR.
- Scanning Restaurant A marks only Restaurant A's order collected.
- Scanning Restaurant A does not collect Restaurant B's order.
- The overall QR remains usable for other eligible restaurant orders until they are individually collected or the overall authorization is invalidated.
- An already-collected restaurant order must return an explicit already-collected result and must not be collected again.

44.5 Scan Result States

- **Successful collection:** show the restaurant's permitted order/pickup details and a clear successful collection result.
- **Wrong restaurant:** reject without exposing unrelated restaurant information.
- **Already collected:** reject and clearly state that the restaurant order has already been picked up.
- **Invalid/expired:** reject.
- **Payment not verified:** reject.
- **Cancelled/refunded/no-show:** reject.
- **Unauthorized scanner:** reject without exposing private order data.
- **Temporary failure:** do not mark collected; permit safe retry.

45. Pickup Completion and Operational Queue Lifecycle

Successful QR collection is a fundamental state transition, not merely a visual notification. Once the restaurant order is successfully scanned and marked collected, it must immediately stop appearing in active operational interfaces whose purpose is to show work that still needs to be fulfilled or picked up.

45.1 Vendor Admin

- A collected restaurant order must immediately leave the Vendor Admin's default active/upcoming/uncollected order queue.
- The Vendor Admin must not need to manually delete, archive or refresh the order to remove it from the active operational queue.
- The order remains available in completed/history views, search, reporting and financial records.
- The database record is never deleted merely because it leaves the active queue.
- Live refresh/polling/realtime mechanisms must reconcile the UI with the authoritative collected state.
- A collected order must not reappear in the active queue after browser refresh, navigation or session restart.

45.2 Staff

- Once successfully scanned, the restaurant order must immediately disappear from the Staff member's active operational order list.
- It must not remain in the active queue because the page has not been manually refreshed.
- Staff access remains restricted to the staff role and assigned restaurant; the completed-state behaviour must not accidentally expose broader customer/history data.
- The active Staff interface must represent only operational work that still requires action.

45.3 Customer

- The customer order must remain permanently available in Orders history according to retention rules.
- The current order presentation changes from pickup-required/active to collected/completed.
- The QR presentation must make it clear that the relevant restaurant pickup has already been completed.
- For a multi-restaurant order, the overall order remains visible until all relevant restaurant pickups are completed, with each restaurant order showing its own collection state.

45.4 Visibility Is Not Deletion

The system must strictly distinguish operational visibility from database existence. Removing an order from an active queue means changing its operational state/filter eligibility; it never means deleting the order.

State	Active operational queue	History/reporting
Paid / scheduled / preparing / ready	Visible when relevant to the role	Retained
Collected	Not visible in active queue	Retained
Cancelled / refunded / no-show	Not visible as active fulfilment work	Retained/auditable

46. Basic Product Behaviour Audit

The following behaviours are mandatory baseline functionality for a production-quality ordering platform. They must not be treated as optional enhancements merely because they do not require a dedicated feature page.

46.1 Cart Revalidation

Before payment/order confirmation, the backend must revalidate restaurant availability, product availability, prices, quantities, pickup slot, restaurant operating state and all applicable capacity/cutoff rules. A stale cart must not create an invalid paid order.

46.2 Quantity Controls

Quantity cannot become negative or exceed configured limits. Increasing/decreasing quantity updates the UI correctly, and quantity zero removes the item. Server-side validation remains authoritative.

46.3 Loading States

Major pages and mutations must show loading/progress states and prevent duplicate submissions while an authoritative operation is in progress.

46.4 Empty States

Major lists must have deliberate empty states including no restaurants, no search results, empty cart, no current orders, no past orders, no grievances and no relevant dashboard records.

46.5 Error States

Network, server, validation, payment, permission and unavailable-data errors must produce understandable recovery states rather than blank or frozen screens.

46.6 Duplicate-Action Protection

Repeated clicks/taps on Place Order, Pay, Submit Grievance, Save, Payout, Refund, Add Product and similar actions must not create duplicate records. Financial/operational mutations must use appropriate idempotency.

46.7 Refresh/Back Safety

Refresh, browser back navigation, reopening a tab and reconnecting must not duplicate an order/payment/pickup or incorrectly overwrite authoritative state.

46.8 Authoritative Order Status

All roles derive order status from the backend. No role may maintain an independent conflicting status solely in client state.

46.9 Availability/Stock Race Protection

If a product or restaurant becomes unavailable during checkout, the backend prevents invalid confirmation and the customer receives a clear resolution path.

46.10 Restaurant Pause/Closure

Paused, closed or unavailable restaurants cannot accept new paid orders for invalid periods. Existing paid orders remain visible and operationally manageable.

46.11 Pickup Time Consistency

Confirmed pickup time, countdown and grace-period information must use one authoritative timezone and consistent formatting across roles.

46.12 Automatic Ready Transition

If pickup is scheduled for 1:15 PM and the order has not already been marked ready, the system automatically transitions it to ready_for_pickup at 1:20 PM under the configured five-minute rule. This transition is server/time authoritative and idempotent.

46.13 Collected-Order Removal

Successful QR collection immediately removes the restaurant order from active Vendor Admin and Staff queues while preserving it in history and reporting.

46.14 Customer Order History

Past/current orders remain retrievable according to customer permissions, with completed, cancelled, refunded and no-show states distinguishable.

46.15 Profile Persistence

Valid profile changes persist and are reflected consistently wherever the customer profile is displayed.

46.16 Authentication Expiry

Expired sessions safely redirect to authentication without exposing protected data or causing duplicate mutations.

46.17 Authorization Failure

Unauthorized routes, records and actions are denied server-side and do not leak information through URLs, error messages or API responses.

46.18 Search and Filters

Search/filter controls can be cleared/reset and must not display stale results as though they were current authoritative results.

46.19 Pagination/Progressive Loading

Large datasets must load without duplicate or missing records and must maintain stable ordering where applicable.

46.20 Image Fallbacks

Missing or failed restaurant/product images display an intentional fallback rather than broken UI.

46.21 Form Recovery

Validation failures preserve safe user-entered values where appropriate and identify the invalid fields clearly.

46.22 Grievance Context

Starting a grievance from an order automatically links the relevant order, restaurant and available context so the customer does not manually enter identifiers.

46.23 Rating Eligibility

Ratings are accepted only for eligible completed/collected orders and duplicate rating submissions are prevented.

46.24 Archive/Deactivation Safety

Archived/deactivated restaurants, products, staff and vendor accounts do not break historical orders or financial/audit references.

46.25 Financial Consistency

Customer, vendor and Super Admin views use authoritative financial figures and role-appropriate visibility; historical calculations remain stable.

46.26 Connectivity Recovery

Temporary connectivity failure must permit safe retry without assuming that a financial or pickup mutation succeeded.

46.27 Accessibility Basics

Interactive controls have labels, usable focus states, appropriate semantics and sufficient contrast.

46.28 Destructive Action Confirmation

Refund, payout, deactivate, archive, pause and other consequential admin actions require clear confirmation of the exact object/action.

46.29 Notification State

Where notifications exist, read/unread state persists appropriately and is not solely browser-memory state.

46.30 Auditability

Privileged and consequential actions record the actor, timestamp, target and required before/after context according to the existing audit-log requirements.

46.31 Realtime Reconciliation

Where the product uses realtime/polling updates, receiving a state change must reconcile the local interface with the backend rather than blindly append/duplicate records.

46.32 Double-Tab Safety

Opening the same order or scan flow in multiple tabs must not permit conflicting state transitions or bypass server-side validation.

46.33 Session/Role Change Safety

If a user's role, restaurant assignment or account status changes, previously cached privileged UI must not continue granting access; backend authorization remains authoritative.

46.34 URL/Deep-Link Safety

Directly opening an order, restaurant, dashboard or grievance URL must apply the same authorization rules as navigating through the UI.

46.35 Time/Timezone Safety

Server-side timestamps and business-time calculations must use the defined platform timezone rules; client clock manipulation must not make an order ready early or bypass timing restrictions.

47. Mandatory Incoming Developer Verification Gate

The first thing Developers 2, 3 and 4 must do when receiving a project is verify the previous developer's work against the SRS. They must not immediately start their own phase simply because a ZIP, README or completion statement has been received.

- 1. Receive:** obtain the complete source, handover package, environment/configuration templates, migrations, tests, documentation and known-issues register.
- 2. Restore:** run the project and reproduce the handed-over baseline. Developers 2–4 also verify the relevant Supabase state under the V2.3 access structure.
- 3. Audit:** check the previous developer's assigned requirements and phase-completion criteria against the actual implementation.
- 4. Test:** independently execute appropriate functional, permission, database and integration tests. The previous developer's test report is evidence, not automatic acceptance.
- 5. Record:** document every missing, broken, incorrect, insecure, undocumented or SRS-non-compliant item discovered.
- 6. Fix first:** if the previous developer has not completed their SRS obligations, the incoming developer must complete/correct that work before moving to their own phase.
- 7. Re-test:** verify that the corrected previous phase now satisfies its requirements.
- 8. Phase gate:** only then begin the incoming developer's assigned phase.

47.1 Developer 2 Gate

Developer 2 must first verify Developer 1's Phases 1–3. Any incomplete foundation, customer ordering, scheduling, payment/order/QR, authentication, handover or other SRS requirement must be corrected before Developer 2 proceeds with Phases 4–6. This does not mean Developer 2 receives the owner's credentials; the V2.3 Supabase access structure remains unchanged.

47.2 Developer 3 Gate

Developer 3 must first verify Developer 2's Phases 4–6, correct non-compliant work, re-test the baseline and only then begin Phases 7–9.

47.3 Developer 4 Gate

Developer 4 must first verify Developer 3's Phases 7–9, correct non-compliant work, re-test the baseline and only then begin the release-blocking Security and Full Code Debugging phases.

48. Handover Acceptance Record

Before an incoming developer proceeds to their own phase, they must create a Phase-Gate Acceptance Record containing:

- Previous developer and phase range reviewed.
- SRS version used for the audit.
- Requirements/features checked.
- Tests executed and results.
- Defects/non-compliance discovered.
- Corrective work completed.
- Remaining issues, if any, with explicit classification.
- Confirmation that the previous phase now satisfies its SRS completion criteria.
- Date and developer identity of the verification.

49. Updated Final Implementation Control Matrix

Domain	V1 must be true
QR	Generation, display, browser scanning and server validation are UNI8-controlled; no third-party QR SaaS is required.
Pickup	Successful restaurant scan atomically marks that restaurant order collected.
Operational queues	Collected orders immediately disappear from active Vendor Admin and Staff queues and never disappear from the database.
Customer Orders	Collected orders remain in history and show completed/collected state.
Multi-restaurant	One customer QR can be used across restaurants, with each scan exposing and collecting only that restaurant's order.
Basic UX	Loading, empty, error, retry, duplicate-action, refresh/back and connectivity behaviours are implemented.
Integrity	Cart, availability, quantity, status, timing and financial values are revalidated server-side.
Handover	Developers 2–4 verify the previous phase against the SRS and fix non-compliance before starting their own phase.
Supabase	Developer 1 has no project access; owner creates project after Phase 3; Developer 2 is first direct-access developer; Developer

50. V2.4 Non-Removal and Compatibility Rule

All previously approved UNI8 requirements remain active. The in-house QR system replaces only the dependency on third-party QR generation/scanning services; it does not remove QR security, anti-replay, restaurant-scoping, audit or collection requirements. The post-pickup queue rule clarifies operational visibility and does not delete historical order data. The basic-product-behaviour requirements are mandatory V1 quality requirements and must be implemented wherever the corresponding functionality exists.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.5 Orderer Identity Display in Operational Dashboards

This addendum supplements the complete UNI8 V2.4 Master SRS. No existing requirement is removed. It adds a mandatory operational visibility requirement for the customer name associated with active restaurant orders.

51. Customer / Orderer Name in Staff and Vendor Admin Order Interfaces

Where an order appears in the **Staff dashboard** or **Vendor Admin dashboard**, the interface must also display the **name of the customer who placed the order**, subject to the role's existing restaurant-scoping and authorization rules.

51.1 Required Display

- Every active restaurant order card/row/detail shown to Staff or Vendor Admin must include the orderer's customer name.
- The name must be taken from the authoritative customer/order data and must not be manually entered by staff.
- The name must remain associated with the order even if the customer's profile name is later changed; the order should retain an appropriate historical orderer-name snapshot for operational and audit consistency.
- The name should be displayed prominently enough that Staff/Vendor Admin can identify the customer at pickup without searching for another identifier.
- Where the same customer has multiple active orders, each order must still show the name associated with that specific order.
- For multi-restaurant orders, each restaurant's Staff/Vendor Admin interface may show the customer name for the shared customer order, but must continue to expose only that restaurant's authorized order information.

51.2 Where the Name Appears

The customer name must be visible anywhere an active order is presented for operational action, including the main Staff active-order queue, Vendor Admin active-order queue, order detail view and the successful QR scan/pickup result where customer identification is relevant.

51.3 Privacy and Authorization

Displaying the customer's name is an operational requirement and does not grant broader customer-data access. Staff must only see names for orders belonging to their assigned restaurant, and Vendor Admin must only see names for orders within their authorized restaurant scope. Customer contact details and other personal information must not be exposed unless separately permitted by the existing SRS.

51.4 Collected Orders

When an order is successfully collected and removed from the active Staff/Vendor Admin operational queue, its orderer name remains available wherever the completed/historical order is accessible under that role's permissions. The order must not be deleted merely because it was picked up.

51.5 Acceptance Criteria

- An active order in Staff dashboard displays the customer's name.
- An active order in Vendor Admin dashboard displays the customer's name.
- The name is correctly associated with the specific order.
- The name remains available after collection in permitted historical views.
- Staff cannot use this feature to access another restaurant's customer/order data.
- Vendor Admin cannot use this feature to access another vendor's customer/order data.
- The name is not exposed to unauthorized roles or public customer-facing screens merely because it exists on the order.

- QR pickup results may show the customer's name where operationally necessary and permitted, without exposing unrelated restaurant data.

52. Integration with Existing Order Lifecycle

This requirement must integrate with the existing order lifecycle defined in V2.4. The orderer name must be present while the order is active, remain correctly associated through preparation/ready-for-pickup/collection states, and remain available in authorized historical records after the order leaves active operational queues.

This requirement does **not** change the existing QR security model, restaurant-scoped access, multi-restaurant QR behaviour, post-pickup queue removal, customer privacy controls, or Supabase/RLS requirements. The backend remains authoritative for both the order and the identity data displayed with it.

53. V2.5 Master Acceptance Addition

Staff and Vendor Admin operational order interfaces must display the name of the person who placed each order.

This must work consistently across active order lists, order details and applicable pickup confirmation states, while respecting existing role and restaurant access controls.

UNI8 GLOBAL SOFTWARE REQUIREMENTS SPECIFICATION

V2.6 Functional Expansion, Authentication Replacement & Phase Allocation

Cumulative addendum to UNI8 V2.5. Nothing previously approved is removed unless explicitly stated below. Phases 1–3 are already built by Developer 1; therefore every new requirement in this addendum is Phase 4 onward work.

54. Phase Allocation — All New Work Starts From Phase 4

Developer 1's Phases 1–3 are treated as an existing baseline. Developers 2, 3 and 4 must first verify the inherited baseline against the SRS, fix any non-compliance, retest it, and only then proceed with their assigned phase. New functionality below must not be assigned wholesale to Developer 1.

Developer 2 — Phases 4–6

Phase 4: Vendor Admin operational UI, restaurant/product detail and lifecycle behaviour, customer-name display, phone-first dashboard refinement, cart/order operational refinements where shared with Vendor Admin. Phase 5: Staff Orders + Scan, customer-name display, ready-for-pickup controls, QR collection lifecycle and active-queue removal. Phase 6: vendor financial/support work, in-app notification infrastructure and authentication replacement work affecting operational roles. Developer 2 inherits the existing code and makes controlled incremental changes.

Developer 3 — Phases 7–9

Phase 7: Super Admin restaurant states, Inside/Outside University classification, University Place Name, menu/product ordering and visibility controls. Phase 8: Customer 360 and central grievance expansion, including the customer 'food not ready yet' support flow. Phase 9: global authentication migration completion, in-app notification templates/events, source-of-truth/reconciliation controls, audit usability and platform consistency.

Developer 4 — Phases 10–11

Phase 10: security audit of the replacement authentication, QR scanner, university popup, customer-name exposure, notification permissions, RLS and all new flows. Phase 11: full regression of every V2.6 requirement and all related edge cases.

55. Customer Restaurant & Product Experience

The customer restaurant page must explicitly present restaurant identity, image where configured, status, hours, pickup availability, next available pickup, categories, products, product availability and operational notices. Inside-University restaurants trigger the existing mandatory access popup before ordering.

Product detail must show name, image, description, price, availability, cook/preparation time and quantity controls where applicable; add-to-cart feedback must be immediate; unavailable products cannot be purchased.

Cart must support quantity increase/decrease, item removal, restaurant-section clearing and full-cart clearing where applicable. It must group items by restaurant, recalculate totals, preserve safe navigation state and revalidate invalidated selections.

56. Checkout, Payment & Order Recovery

Before payment/order confirmation, the backend must revalidate restaurant state, product availability, prices, quantities, pickup slots, capacity, cutoff, hours and scheduling feasibility. If invalid, payment/order creation is blocked and the customer receives a clear recovery action.

Payment failure must provide safe retry/return behaviour without duplicate orders. If payment may have succeeded but confirmation is pending, show a pending/reconciliation state instead of falsely declaring failure. Successful payment must produce a clear confirmation screen with order ID, restaurant(s), items, total, pickup time(s), status and valid QR access.

57. Order Timeline & Status Ownership

Transition	Authority
payment_pending → paid	Verified payment/backend
paid → scheduled	Backend scheduling
scheduled → preparing	Staff or authorized Vendor Admin
preparing → ready_for_pickup	Staff or authorized Vendor Admin, or automatic server T+5 if not already ready
ready_for_pickup → collected	Successful authorized QR scan only
Eligible → no_show	System/authorized operational process
Refund pending → refunded	Authorized refund/payment confirmation

Customer order details must show a clear timeline: order placed/payment confirmed → preparing → ready → collected, with applicable timestamps. No role gets an unrestricted status dropdown.

58. Ready-for-Pickup Controls

Both Vendor Admin and Staff must be able to mark an eligible order **Ready for Pickup**. The action is server-authorized, restaurant-scoped, audit logged and idempotent. If nobody manually marks it ready, the existing server-side automatic T+5 rule applies. If it was manually marked ready earlier, the automatic job must not make a conflicting transition.

59. Customer 'Food Not Ready Yet' Grievance UX

If the customer is shown that an order is Ready for Pickup but it has not been collected and the applicable five-minute threshold has elapsed, the customer dashboard must present a polished, prominent in-app prompt explaining that the system shows the order as ready while the food may not actually be ready.

- Provide a clear primary 'Raise a Ticket' action and an appropriate secondary action.
- Opening the ticket automatically links customer, order and restaurant context.
- The customer does not manually enter the order ID.
- The ticket routes only to UNI8 Super Admin/support.
- The prompt does not automatically declare the restaurant at fault.
- Duplicate automatic tickets must be prevented.
- The customer can follow the resulting ticket through the central grievance CRM.

60. Restaurant & Product Lifecycle

- Restaurant states are explicitly Open, Paused, Closed and Archived. Paused/Closed prevents new invalid orders while existing paid orders remain manageable; Archived removes normal discovery but preserves history.
- Product active/archived, visibility and stock/availability are distinct concepts where required.
- Historical order item snapshots remain stable when product details change.
- Products referenced by historical orders must not be hard-deleted in a way that breaks history.
- Vendor Admin/Super Admin can control product display order within categories.
- Category/product ordering is deterministic and persists.

61. Staff & Vendor Admin Operational Views

- Staff active orders show customer name, order ID, items, quantity, pickup time and operational status within existing permissions.
- Vendor Admin active orders show customer name, order ID, items, pickup time and operational status.
- Both roles can mark eligible orders Ready for Pickup.

- Successful QR collection immediately removes that restaurant order from active operational queues without manual deletion or refresh.
- Collected orders remain in permitted history/reporting.
- Customer name visibility never grants access to unrelated restaurants or broader customer CRM data.

62. Customer Authentication — OTP Removed From Final V1

OTP is removed as the final V1 customer login method. Phases 1–3 already contain the earlier OTP implementation, so Phase 4 onward must replace/remove the active OTP customer-login experience.

- During signup, customer provides the required profile information, including phone number and email, and creates a password.
- Phone number and/or email is the customer login ID according to the final implemented login configuration.
- Customer subsequently logs in using the configured ID and password.
- Passwords are managed by the authentication system/provider and never stored in plaintext by UNI8.
- Secure password recovery/reset is required.
- Login/session abuse protection, rate limiting, session security and force logout remain mandatory.
- OTP-only screens, resend flows and OTP login paths must not remain active in final V1.
- Existing Phase 1–3 OTP code may be retained only as migration/history support and must not be the final customer authentication path.
- Phase 10 specifically audits the replacement credential flow for brute force, credential stuffing, reset abuse, session fixation and account takeover.

63. In-App Notifications Only

V1 uses **in-app notifications only**. SMS notification delivery is not part of the current V1 notification layer.

- Notifications are stored server-side with user/order/ticket/event linkage and persistent read/unread state.
- Customer events may include order confirmation, payment state, ready-for-pickup, collection, refund and support updates.
- Vendor/Staff events cover authorized operational events.
- Super Admin events cover authorized operational alerts, grievances and payment exceptions.
- Notification failure never changes underlying order/payment/pickup state.
- Role and restaurant scoping applies to notifications.
- Super Admin-controlled templates remain supported where defined by the platform settings.

64. Global Source-of-Truth Rule

Frontend state is never authoritative for critical business state. Backend/database state, plus verified Razorpay state for payment, controls the following:

Domain	Authority
Order status	UNI8 backend/database
Payment	Verified Razorpay + UNI8 records
Prices/totals	Server calculation
Commission	Database configuration + order snapshot
Pickup time/capacity/cutoff	Backend
Availability	Backend/database

Domain	Authority
QR validity/collection	UNI8 backend/database
Role/scope	Auth + RBAC/RLS
Refund	Financial records + payment confirmation
Vendor payable	Financial records
Grievance	Central CRM
Notifications	Server-side notification records

65. Search, Filters, Sessions & Safety Basics

- Search is case-insensitive with relevant partial matching; operational search can include order ID/customer name/product/restaurant/status/date as permitted by role.
- Large lists use server-side filtering and pagination; clear/reset controls exist.
- Customer has clear logout; expired sessions redirect safely.
- Disabled users cannot continue privileged access through cached screens.
- Protected deep links enforce the same server-side authorization as normal navigation.
- Refund, payout, deactivate, archive, pause, role change and staff-removal actions require explicit confirmation.
- Historical records remain intact after deactivation/archive.
- Multi-tab use cannot create conflicting order/payment/pickup transitions.
- Time calculations use authoritative server/platform timezone rules; client clock manipulation cannot bypass pickup timing.
- Connectivity failure must allow safe retry without assuming an authoritative mutation succeeded.

66. Financial Edge Cases

- V1 refund policy must explicitly enforce whether refunds are full-only or allow partial refunds; the UI/backend must not invent arbitrary behaviour.
- If a refund occurs after a vendor payable is created or disbursed, create an auditable reversal/adjustment rather than silently rewriting history.
- Vendor payable, commission, refund and disbursement records remain traceable to the originating order and financial events.
- Super Admin can identify unresolved financial exceptions requiring manual action.

67. Multi-Restaurant Partial Completion

- Each restaurant order in a multi-restaurant checkout has an independent operational collection state.
- Collecting Restaurant A does not collect Restaurant B.
- Customer sees the overall order and each restaurant's individual state.
- If one restaurant becomes unable to fulfil after another is collected, only the affected restaurant order enters the applicable cancellation/refund/grievance process.
- Any resulting refund or payable adjustment is an auditable financial event.

68. Cart Persistence

The customer cart persists indefinitely on the same browser until the cart is changed, completed/consumed by an order, explicitly cleared or invalidated by an authoritative rule.

- Browser refresh preserves the cart.
- Closing and reopening the same browser profile preserves the cart.
- Persistence never bypasses server-side checkout revalidation.
- Invalid persisted products/restaurants/pickup selections are clearly identified and cannot be paid for until resolved.

69. Final V2.6 Acceptance Criteria

- All V2.6 requirements are implemented from Phase 4 onward.
- Developers 2, 3 and 4 verify inherited work against the SRS before their phase and fix non-compliance first.
- Vendor Admin and Staff can mark eligible orders Ready for Pickup.
- Customer sees an order timeline and authoritative status transitions.
- Customer receives a polished in-app 'food not ready yet' grievance prompt after the defined five-minute condition.

- OTP is removed from final V1 customer login and replaced by ID/password authentication.
- V1 notifications are in-app only.
- Restaurant detail, product detail and cart editing are fully functional.
- Collected orders disappear from active Vendor Admin/Staff queues while remaining in history.
- Customer name is visible on operational Staff/Vendor Admin orders within scope.
- QR has no fixed time expiry; after successful pickup, the customer website no longer displays that QR anywhere. A screenshot/reuse attempt returns an already-collected/invalid result and never recollects the order.
- Financial, multi-restaurant, authentication, connectivity and stale-state edge cases are handled.
- Phase 10 and Phase 11 fully security-test and regression-test all V2.6 additions.

70. Non-Removal Rule

All earlier SRS requirements remain active. V2.6 specifically changes the final customer authentication method from OTP to ID/password, changes V1 notifications to in-app only, and adds/refines the behaviours described above. These changes do not authorize removal of unrelated existing security, QR, payment, grievance, RBAC, Supabase, audit or handover requirements.